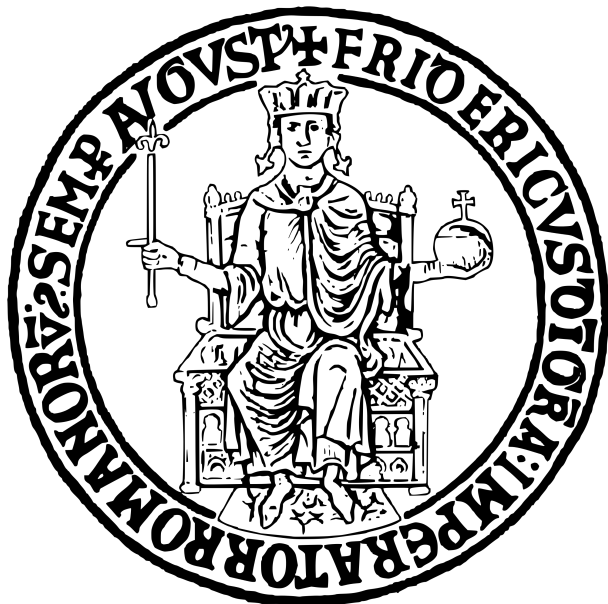


Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica



Prof.ssa
Anna Rita Fasolino
a.a. 2025-26

Team D8:
Auricchio Martina DE9000125
Cafora Ciro DE9000117
Picariello Ilaria M63001580
Vollaro Fabrizio M63001569

Indice

1	Introduzione	1
1.1	Identificazione del Team	1
1.2	Riferimenti ai Repository	1
1.3	Descrizione del Task Assegnato	1
1.4	Approccio di Sviluppo e Gestione del Progetto	2
1.4.1	Stato Iniziale dell'Architettura (AS-IS)	3
1.4.2	Evoluzione Architetture e Introduzione Microservizio T9 (TO-BE)	5
1.5	Analisi dell'Impatto	7
2	Analisi dei Requisiti	8
2.1	Documenti dei Requisiti	8
2.2	Specifica dei Requisiti Funzionali	9
2.3	Requisiti Non Funzionali	11
2.3.1	Requisiti del Prodotto (Qualità del Software)	11
2.3.2	Requisiti Organizzativi (Sviluppo e Standard)	11
2.3.3	Requisiti Esterni (Sicurezza e Privacy)	11
2.4	Diagramma delle Classi : T9	12
2.5	Diagrammi dei Requisiti	13
2.5.1	Use Case Diagram	13
3	Progettazione della Soluzione	22
3.1	View dei Componenti e Interfaccia REST API	22
3.1.1	Scelte architetture	22
3.1.2	Vista a Componenti e Interazioni tra i Servizi	22
3.1.3	Interfaccia REST del microservizio T9	24
3.1.4	Ricerca del profilo utente e integrazione della componente social	24
3.1.5	Integrazione tramite API Gateway	25
3.2	View Modulare	25
3.2.1	Package Diagram del microservizio T9	25
3.3	View Dinamica	26
3.3.1	Sequence Diagram	26

3.4	Prototipo dell'Interfaccia Utente	31
3.4.1	Obiettivi di progettazione della UI	31
3.4.2	Activity Diagram	31
3.4.3	Prototipo UI	36
3.5	Deployment Diagram dei Microservizi Coinvolti	38
3.6	Progettazione del Database	39
3.6.1	Database del microservizio T9	39
3.6.2	Modello Entity-Relationship	39
3.6.3	Struttura della tabella Profile	40
3.6.4	Ruolo del microservizio T23 nella gestione social	40
3.7	Issue e Anti-pattern	40
4	Implementazione e Struttura del Progetto	42
4.1	Elenco Moduli Aggiunti o Modificati	43
4.2	Close-up sui File Principali	44
4.2.1	T9 – UserProfileBioService.java	44
4.2.2	T9 – PlayerBioDataService.java	45
4.2.3	T9 – GroqLlmClient.java	46
4.2.4	T9 – UserProfileService.java (Nickname uniqueness)	48
4.2.5	T9 – RestExceptionHandler.java (HTTP 409)	48
4.2.6	T9 – UserProfileController.java	49
4.2.7	T23 – PlayerController.java	49
4.2.8	T23 – UserProfileRepository.java	50
4.2.9	T23 – PlayerService.java	50
4.2.10	T23 – AuthService.java	51
4.2.11	T5 – UserProfileController.java	51
4.2.12	T5 – friend_profile.js	52
4.2.13	T5 – profile.js	52
4.2.14	T5 – fragments/Navbar.html (link alla pagina profilo)	53
4.2.15	T5 – friend_profile.html	53
4.2.16	T5 – t9_profile.html	53
5	Test	54
5.1	Ambito dei Test	54
5.2	Strumenti Utilizzati	54
5.3	Casi di Test Principali	55
5.3.1	Test API – Microservizio T9	55
5.3.2	Unit Test	55
5.3.3	Test di Integrazione – T23 → T9	56

5.3.4	Test di Integrazione – T9 $\rightarrow$ LLM	56
5.3.5	Test End-to-End	56
5.4	Problemi Rilevati e Correzioni	57
5.5	Limitazioni Note	57
6	Sviluppi Futuri	59

Capitolo 1

Introduzione

1.1 Identificazione del Team

- **ID Team:** D8
- **Componenti del Gruppo:**
 - Martina Auricchio – `marti.auricchio@studenti.unina.it`
 - Ciro Cafora – `ci.cafora@studenti.unina.it`
 - Ilaria Picariello – `il.picariello@studenti.unina.it`
 - Fabrizio Vollaro – `fab.vollaro@studenti.unina.it`

1.2 Riferimenti ai Repository

- **URL Versione di Partenza:** `https://github.com/Testing-Game-SAD-2023/A13`
- **URL Repository Progetto Consegnato:** `https://github.com/Testing-Game-SAD-2023/A13/tree/D8_TaskR6_ProfiloGiocatore`

1.3 Descrizione del Task Assegnato

Il task assegnato ha riguardato la progettazione e l'implementazione di una **pagina dedicata al profilo del giocatore**, con l'obiettivo di migliorare l'organizzazione, l'accessibilità e la coerenza delle informazioni personali all'interno della Web Application.

Nella versione iniziale del sistema non era presente una vera e propria pagina di profilo centralizzata. Il microservizio **T23 (User Service)** era principalmente responsabile della registrazione e dell'autenticazione degli utenti, occupandosi della gestione delle credenziali e della validazione dei token di accesso, ma non forniva una rappresentazione

strutturata del profilo del giocatore. Alcune informazioni dell'utente risultavano accessibili solo indirettamente tramite elementi dell'interfaccia presenti nella homepage, gestita dal microservizio **T5 (Game Front End / UI Service)**, senza una vista organica e unificata.

Questa configurazione comportava una fruizione frammentata delle informazioni personali e una scarsa chiarezza nella distinzione tra i dati legati all'autenticazione e quelli relativi all'identità applicativa del giocatore. Di conseguenza, l'esperienza d'uso risultava poco coerente e difficilmente estendibile.

L'intervento progettuale ha quindi avuto come obiettivo principale l'introduzione di una **pagina di profilo strutturata e autonoma**, progettata per presentare le informazioni dell'utente in modo chiaro, coerente e facilmente estendibile. La soluzione adottata segue i principi di **separazione delle responsabilità** e **modularità** propri dell'architettura a microservizi, incapsulando le nuove funzionalità all'interno di un **nuovo microservizio dedicato**, denominato **T9 Profile Service**.

Sono stati inoltre introdotti elementi di personalizzazione del profilo e funzionalità interattive, con l'obiettivo di **migliorare l'engagement dell'utente** senza compromettere la semplicità dell'interfaccia. In particolare, il task ha incluso l'implementazione di una **componente social** e di un sistema automatico di **generazione della biografia del giocatore tramite tecniche di Intelligenza Artificiale**, concepiti come estensioni funzionali del profilo utente.

Nel complesso, il task ha portato alla definizione di una gestione del profilo più chiara e strutturata, ponendo le basi per future evoluzioni dell'interfaccia e delle funzionalità legate all'identità del giocatore.

1.4 Approccio di Sviluppo e Gestione del Progetto

Nelle fasi iniziali del progetto è stato adottato un approccio di tipo **top-down**, finalizzato alla scomposizione del problema complessivo in sotto-componenti funzionalmente indipendenti. Tale strategia ha guidato una prima analisi delle funzionalità richieste e della loro possibile integrazione all'interno dell'architettura esistente. In questo contesto, si è inizialmente tentato di intervenire direttamente sul microservizio **T23 (User Service)**, o su una sua copia, al fine di sviluppare e testare le nuove funzionalità in modo isolato.

Questo approccio si è tuttavia rivelato poco efficace. Il microservizio T23 risulta infatti fortemente accoppiato ai meccanismi di autenticazione, alla gestione del database e ai flussi di login dell'utente. Tali dipendenze hanno generato conflitti e incoerenze durante i tentativi di estensione, evidenziando come le criticità non fossero riconducibili alla sola implementazione di nuove funzionalità, ma coinvolgessero la struttura architettonica complessiva del sistema.

Di conseguenza, è stato adottato un approccio **bottom-up**, concentrandosi inizialmente sulla comprensione dei componenti esistenti e delle loro interazioni, per poi costruire progressivamente le nuove funzionalità a partire dai servizi già stabili e consolidati. Lo sviluppo è stato organizzato secondo un **modello iterativo e incrementale**, in cui ogni iterazione ha prodotto un incremento funzionale autonomo, verificabile e facilmente integrabile nel sistema complessivo.

L'approccio seguito si ispira ai principi Agile e, in particolare, a una modalità di lavoro affine a **Scrum**, seppur in forma semplificata. Il lavoro è stato suddiviso in brevi cicli di sviluppo, ciascuno caratterizzato da obiettivi chiari, integrazione continua delle modifiche e frequenti momenti di verifica dello stato del sistema. Questa impostazione ha favorito flessibilità e capacità di adattamento, consentendo di affrontare in modo efficace le criticità emerse durante lo sviluppo.

1.4.1 Stato Iniziale dell'Architettura (AS-IS)

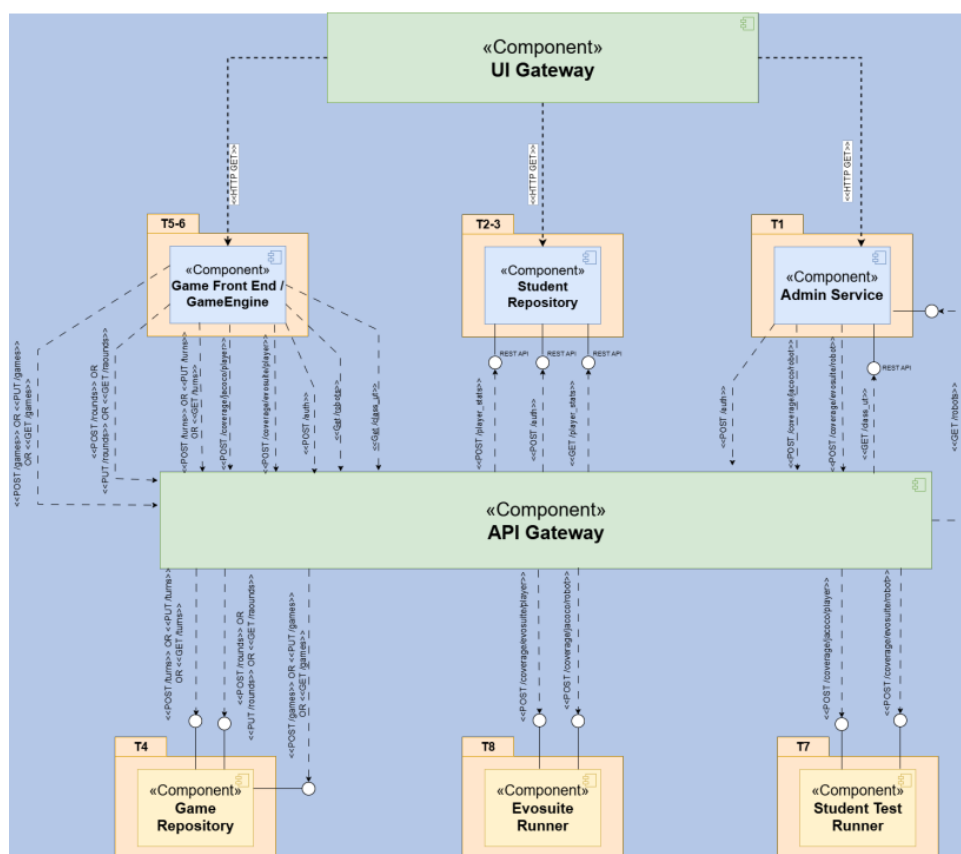


Figura 1.1: Architettura AS-IS

Nella configurazione iniziale, l'architettura del sistema presentava una gestione dell'identità del giocatore distribuita su più microservizi, senza un componente chiaramente dedicato alla rappresentazione e alla gestione completa del profilo utente.

In particolare, lo stato iniziale (“As-Is”), come descritto nella presentazione del task e rappresentato nella vista C&C iniziale, era caratterizzato dai seguenti aspetti:

- **Separazione tra autenticazione e interfaccia utente:** Le informazioni anagrafiche di base e i meccanismi di autenticazione (registrazione, login, gestione dei token) erano gestiti dal microservizio **T23 (User Service)**, il cui ruolo era limitato alla sicurezza e all’identificazione dell’utente.
- **Assenza di una pagina di profilo dedicata:** Non era presente una vista unificata del profilo del giocatore. Le informazioni disponibili all’utente erano accessibili solo indirettamente tramite la homepage e altre sezioni dell’interfaccia, gestite dal microservizio **T5 (Game Front End / UI Service)**.
- **Mancanza di una gestione strutturata del profilo:** L’architettura non prevedeva un microservizio responsabile della gestione di dati quali nickname, biografia, preferenze o impostazioni di visibilità, rendendo difficile l’estensione funzionale dell’identità del giocatore.

In questa configurazione, il microservizio T23 svolgeva esclusivamente il ruolo di servizio di autenticazione e gestione delle credenziali, senza essere concepito per supportare funzionalità di dominio legate alla rappresentazione del profilo utente. I tentativi di estendere direttamente tale servizio hanno evidenziato limiti strutturali dovuti all’elevato accoppiamento con i meccanismi di sicurezza e alla mancanza di una chiara separazione delle responsabilità.

Queste criticità hanno reso evidente la necessità di un’evoluzione architetturale, orientata all’introduzione di un componente dedicato alla gestione del profilo del giocatore.

1.4.2 Evoluzione Architeturale e Introduzione Microservizio T9 (TO-BE)

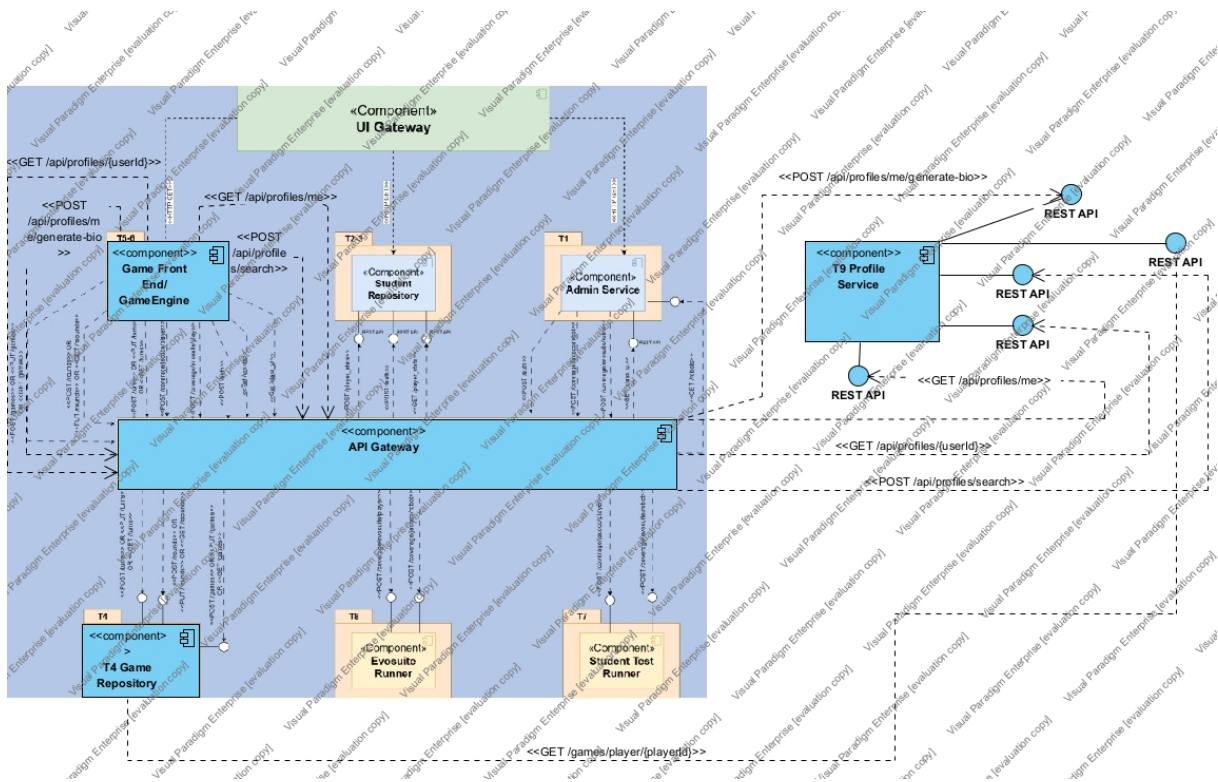


Figura 1.2: System Architecture – C&C View

Per superare i limiti strutturali emersi nello stato iniziale, l'architettura è stata evoluta introducendo un nuovo componente autonomo: il **microservizio T9 (Profile Service)**. Questa scelta ha consentito di separare in modo chiaro le responsabilità legate all'autenticazione da quelle relative alla gestione del profilo utente.

Il microservizio T9 rappresenta ora l'unica *source of truth* per l'identità applicativa del giocatore, centralizzando la gestione delle informazioni di profilo e abilitando una vista unificata dell'utente. In particolare, l'evoluzione architeturale ha introdotto le seguenti modifiche:

- **Centralizzazione del profilo utente:** T9 gestisce in modo esclusivo dati quali nickname, biografia, preferenze e metadati del profilo, separandoli dalle informazioni di autenticazione mantenute in T23.
- **Database dedicato:** Il microservizio T9 dispone di un database isolato, riducendo l'accoppiamento con i servizi legacy e permettendo un'evoluzione indipendente dello schema dati.
- **Integrazione tramite API Gateway:** Tutte le interazioni con T9 avvengono attraverso l'API Gateway, che espone il path `/api/profiles/**` e instrada le richieste verso il servizio. L'accesso è protetto dai meccanismi di autenticazione già esistenti.

Dal punto di vista dei flussi di comunicazione, come illustrato nel diagramma C&C, il microservizio T9 viene invocato dal frontend (**T5**) tramite l'API Gateway attraverso chiamate REST di tipo GET e POST. In particolare:

- le richieste GET `/api/profiles/me` e GET `/api/profiles/{userId}` consentono il recupero delle informazioni di profilo;
- la richiesta POST `/api/profiles/search` permette la ricerca di profili utente;
- la richiesta POST `/api/profiles/me/generate-bio` abilita la generazione automatica della biografia tramite servizi esterni di Intelligenza Artificiale.

L'API Gateway svolge il ruolo di punto di ingresso unico, occupandosi dell'autenticazione (in collaborazione con T23) e del routing delle richieste verso T9. Il microservizio T23 rimane quindi confinato alla gestione della sicurezza, senza introdurre dipendenze dirette con T9.

Infine, T9 interagisce con il microservizio **T4 (Game Repository)** tramite una chiamata GET `/games/player/{playerId}`, al fine di arricchire il profilo utente con informazioni relative ai progressi di gioco. Questa dipendenza *service-to-service* consente di mantenere separati i domini applicativi, preservando al contempo una visione coerente e completa del giocatore all'interno del sistema.

1.5 Analisi dell’Impatto

Tabella 1.1: Analisi dell’impatto sui componenti architetturali

Componente	Tipo di Intervento	Descrizione dell’Impatto	Livello di Impatto
T9 (Profile Service)	Nuovo micro-servizio	Microservizio sviluppato interamente da zero per la gestione del profilo utente. Include persistenza dedicata, API REST per il profilo, generazione automatica della biografia tramite intelligenza artificiale e integrazione con altri servizi per l’arricchimento dei dati.	Alto
T23 (User Service)	Modifica limitata	Estensione del flusso di registrazione e login per inizializzare e sincronizzare il profilo utente su T9 tramite chiamate REST. Le funzionalità principali di autenticazione e social graph rimangono invariate.	Basso
T5 (UI Service)	Modifica funzionale	Aggiornamento dell’interfaccia utente con introduzione di una pagina di profilo unificata e modifica della navbar. Aggregazione dei dati di profilo e social tramite chiamate ai microservizi T9 e T23, senza modifiche alla logica di business.	Medio
API Gateway	Modifica configurativa	Aggiornamento del file <code>application.yml</code> per introdurre il routing verso il nuovo microservizio T9. Nessuna modifica alla logica applicativa o ai meccanismi di sicurezza del gateway.	Basso
Database T9	Nuova istanza	Database dedicato al microservizio T9 per la persistenza delle informazioni di profilo. Garantisce isolamento dei dati e indipendenza dagli altri microservizi.	Medio
Database T23	Nessuna modifica strutturale	Il database esistente di T23 non subisce modifiche strutturali. Continua a gestire dati di autenticazione, progressi e social graph senza impatti diretti.	Nulla

Capitolo 2

Analisi dei Requisiti

2.1 Documenti dei Requisiti

Questa sezione definisce i servizi richiesti dal punto di vista dell'utente finale, indipendentemente dall'implementazione tecnica. Ogni requisito utente è identificato da un codice univoco (RU).

RU-01: Gestione del Profilo Il sistema deve garantire che ogni utente registrato disponga di un profilo giocatore sincronizzato, senza richiedere interventi manuali.

RU-02: Visualizzazione Centralizzata L'utente deve poter consultare tutte le informazioni relative al proprio account (dati, statistiche, preferenze) in un'unica vista aggregata.

RU-03: Personalizzazione Identità Il sistema deve consentire all'utente di modificare i propri dati visibili (nickname, biografia) e l'immagine rappresentativa (avatar) per esprimere la propria identità di gioco.

RU-04: Generazione Biografia L'utente deve poter usufruire di un assistente intelligente per la generazione automatica di una biografia coerente con le proprie statistiche di gioco.

RU-05: Configurabilità Interfaccia L'utente deve avere la possibilità di adattare l'aspetto grafico dell'applicazione e le impostazioni di privacy alle proprie preferenze.

RU-06: Interazione Social Il sistema deve permettere agli utenti di esplorare la community visualizzando i profili degli altri giocatori e di gestire una propria rete di contatti tramite funzionalità di "follow" e "unfollow".

2.2 Specifica dei Requisiti Funzionali

Di seguito sono descritti nel dettaglio i servizi che il sistema deve fornire, specificando input, elaborazione e output attesi. Ogni requisito funzionale è identificato da un codice univoco (**RF**).

RF-01: Inizializzazione Profilo • **Descrizione:** Il sistema deve creare un'istanza del profilo giocatore nel momento in cui viene notificata la registrazione di un nuovo utente dal servizio di autenticazione.

- **Input:** Identificativo utente (`userId`), Email, Nome (ricevuti dal microservizio T23).
- **Elaborazione:**
 1. Ricezione della chiamata sull'endpoint `/init`.
 2. Verifica dell'esistenza del profilo (idempotenza).
 3. Se il profilo non esiste, creazione di un nuovo record con impostazioni di default (avatar predefinito, tema base).
- **Output:** Conferma dell'avvenuta creazione.

RF-02: Recupero Profilo Completo • **Descrizione:** Il sistema deve consentire il recupero di tutte le informazioni aggregate del profilo utente tramite un'unica interrogazione per ottimizzare il caricamento lato frontend.

- **Input:** Identificativo utente (`userId`) estratto dal token di sessione.
- **Elaborazione:** Interrogazione del database T9 per recuperare: dati anagrafici, biografia, percorso avatar e preferenze UI.
- **Output:** Oggetto JSON (DTO) contenente l'intero profilo utente.

RF-03: Gestione Dati Anagrafici (Upsert) • **Descrizione:** Il sistema deve permettere la modifica dei dati anagrafici e della biografia, gestendo automaticamente sia l'inserimento (se il dato manca) sia l'aggiornamento.

- **Input:** Oggetto JSON con i campi da modificare (es. `nickname`, `bio`).
- **Elaborazione:**
 1. Validazione dei dati in ingresso.
 2. Verifica unicità del `nickname` (se modificato).
 3. Aggiornamento del record nel database secondo logica *upsert*.
- **Output:** Dati del profilo aggiornati.

RF-04: Aggiornamento Avatar • **Descrizione:** Il sistema deve fornire una funzionalità dedicata esclusivamente alla modifica dell'immagine rappresentativa dell'utente.

- **Input:** Stringa contenente il percorso della nuova immagine.

- **Elaborazione:** Aggiornamento puntuale del campo `profilePicturePath` senza sovrascrivere altri dati del profilo.
- **Output:** Conferma di avvenuto aggiornamento.

RF-05: Generazione Automatica Biografia (AI) • **Descrizione:** Il sistema deve generare una descrizione testuale del giocatore utilizzando un modello linguistico (LLM) esterno, basandosi sui dati di gioco.

- **Input:** Dati attuali del profilo (nickname, statistiche di gioco).
- **Elaborazione:**
 1. Costruzione di un prompt contestualizzato con i dati utente.
 2. Invio della richiesta al servizio LLM esterno.
 3. Ricezione, validazione e troncamento della risposta per rispettare i limiti di caratteri.
 4. Salvataggio automatico della bio generata nel database.
- **Output:** Testo della biografia generata.

RF-06: Personalizzazione Interfaccia • **Descrizione:** Il sistema deve memorizzare le preferenze utente relative alla visualizzazione dell'interfaccia.

- **Input:** Preferenze di tema (colori), lingua e impostazioni di privacy (profilo pubblico/privato).
- **Elaborazione:** Persistenza delle configurazioni nel profilo utente.
- **Output:** Conferma aggiornamento preferenze.

RF-07: Visualizzazione Profilo Pubblico (Guest View) • **Descrizione:** Il sistema deve esporre un endpoint pubblico per recuperare le informazioni non sensibili di un altro giocatore (Nickname, Avatar, Biografia).

- **Input:** ID dell'utente target (`targetUserId`).
- **Elaborazione:**
 1. Ricezione della richiesta di lettura per un ID specifico.
 2. Recupero dei dati dal DB.
 3. Filtraggio dei dati sensibili (es. email, preferenze UI) per esporre solo il DTO pubblico.
- **Output:** Ottenimento profilo.

RF-08: Gestione Relazioni (Follow/Unfollow) • **Descrizione:** Il sistema deve gestire la creazione e la rimozione di relazioni di "seguito" tra due utenti, aggiornando i contatori visibili sui rispettivi profili.

- **Input:** ID dell'utente target e tipo di azione (Follow/Unfollow).
- **Elaborazione:**
 1. Verifica validità token richiedente.

2. Inoltro della richiesta al servizio di gestione relazioni.
 3. Aggiornamento dei contatori "Follower" e "Following" associati ai profili coinvolti.
- **Output:** Conferma operazione e stato aggiornato della relazione

2.3 Requisiti Non Funzionali

I requisiti non funzionali definiscono le proprietà del sistema e i vincoli operativi, classificati secondo la tassonomia standard (Requisiti di Prodotto, Organizzativi ed Esterni).

2.3.1 Requisiti del Prodotto (Qualità del Software)

- **Affidabilità (Reliability):** Il sistema deve garantire la disponibilità del profilo anche in caso di fallimento temporaneo della fase di registrazione, implementando una strategia di *Lazy Creation* alla prima lettura.
- **Robustezza:** Il sistema deve gestire i fallimenti del servizio esterno LLM (es. timeout) fornendo una risposta di fallback senza bloccare l'applicazione.

2.3.2 Requisiti Organizzativi (Sviluppo e Standard)

- **Manutenibilità:** Il microservizio deve essere sviluppato come unità indipendente con database dedicato, per permettere evoluzioni dello schema dati senza impattare altri servizi.

2.3.3 Requisiti Esterni (Sicurezza e Privacy)

- **Sicurezza (Security):** L'accesso agli endpoint di modifica del profilo deve essere consentito solo a richieste autenticate tramite token JWT validato dall'API Gateway.
- **Privacy:** Il sistema deve rispettare la preferenza dell'utente sulla visibilità del profilo (pubblico/privato) e proteggere i dati personali nel database.

2.4 Diagramma delle Classi : T9

Il diagramma delle classi del microservizio T9 descrive la struttura statica del *Profile System*, evidenziando le principali classi coinvolte, i loro attributi, le operazioni offerte e le relazioni esistenti tra di esse. Il diagramma è stato progettato seguendo i principi di separazione delle responsabilità e modularità tipici dell'architettura a microservizi.

Il diagramma fornisce una visione complessiva dell'organizzazione interna del microservizio dedicato alla gestione del profilo, facilitando la comprensione delle responsabilità delle singole componenti e supportando l'evoluzione futura del sistema.

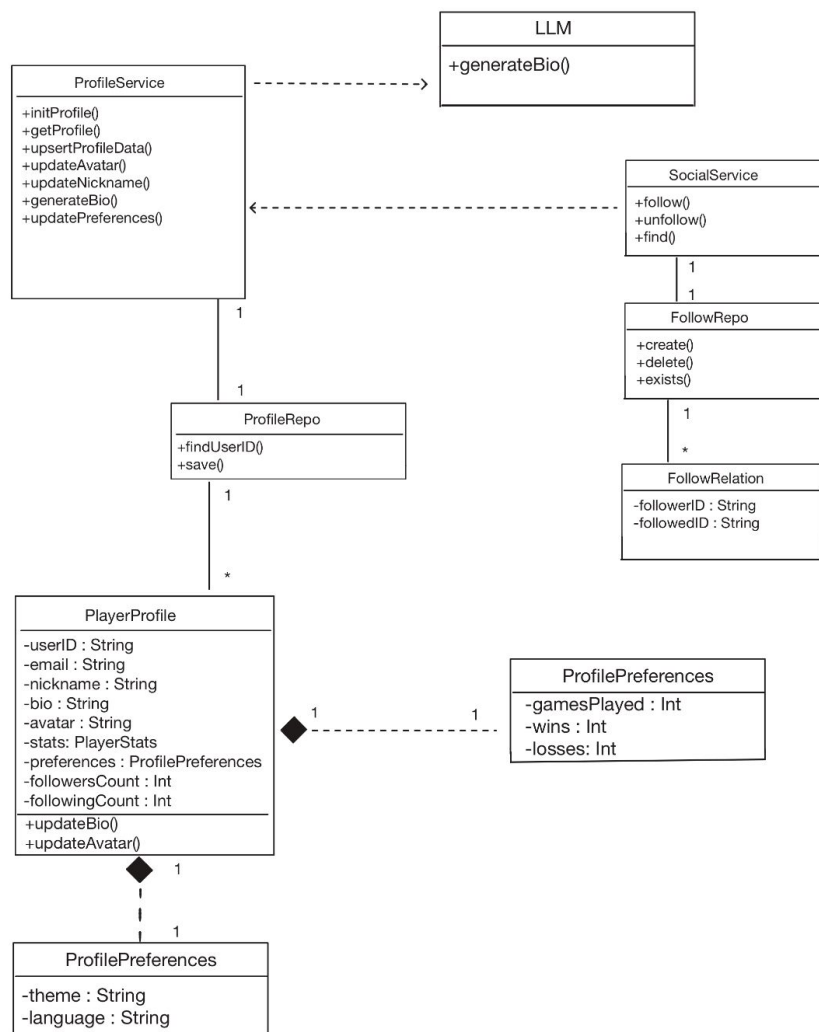


Figura 2.1: Diagramma delle Classi

2.5 Diagrammi dei Requisiti

2.5.1 Use Case Diagram

Partendo dal documento dei requisiti, è stato realizzato un Diagramma dei Casi d'Uso. L'obiettivo è avere una descrizione visuale di:

- Use Case
- Scenari
- Storie Utente

Il diagramma dei casi d'uso rappresenta le principali funzionalità offerte dal *Profile System*, evidenziando le operazioni che il giocatore può svolgere nella gestione del proprio profilo. Abbiamo realizzato una tabella contenente gli scenari relativi al giocatore, che saranno poi mostrati graficamente attraverso il diagramma.

ID Scenario	Attore Primario	Descrizione scenario
S1	Player	Il giocatore visualizza il proprio profilo
S2	Player	Il giocatore modifica le informazioni del profilo
S3	Player	Il giocatore personalizza l'interfaccia della pagina profilo
S4	Player	Il giocatore segue o smette di seguire un altro giocatore

Tabella 2.1: Elenco degli scenari d'uso

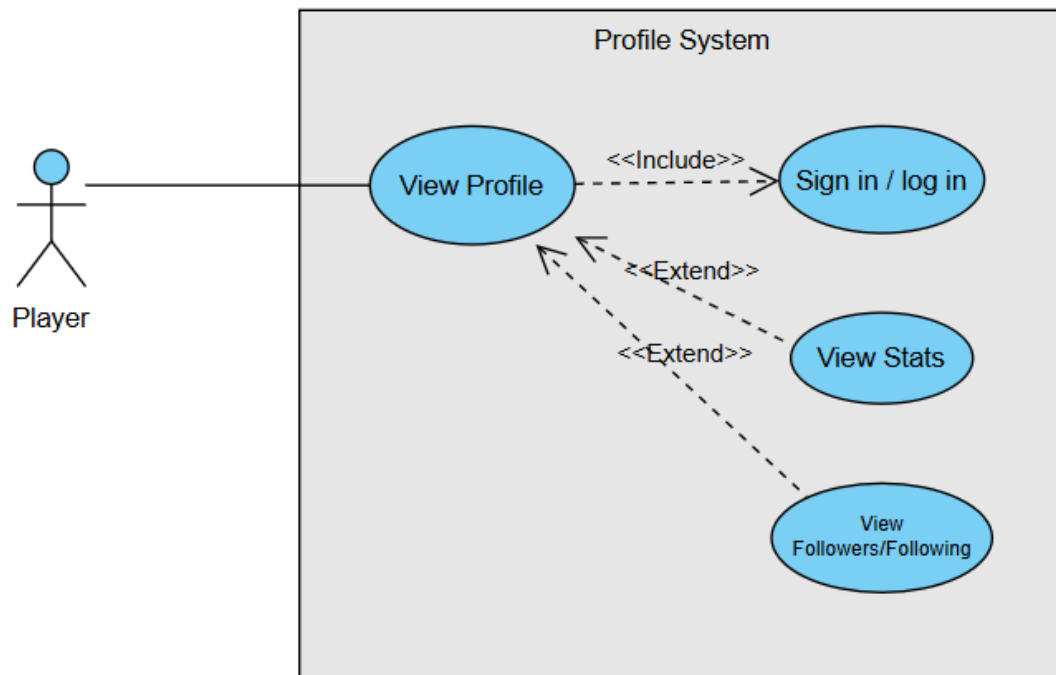


Figura 2.2: Use Case Diagram S1

Campo	Descrizione
ID	UC1
Nome	View Profile
Attore primario	Player
Attore secondario	–
Descrizione	Il sistema mostra il profilo del Player, includendo statistiche e informazioni followers/following quando disponibili.
Relazione	«include» Sign in / log in «extend» View Stats «extend» View Followers/Following
Precondizioni	Il Player ha un account; se non autenticato deve effettuare login o register.
Postcondizioni	La pagina profilo viene renderizzata (con fallback su stats e social se i servizi non sono disponibili).
Flusso principale	1. Il Player inserisce le credenziali (login) oppure si registra (register). 2. Il sistema invia POST /auth/login o /auth/register a T23. 3. Se OK, T23 restituisce JWT (cookie "jwt") e userId. 4. Il Browser carica la pagina profilo con GET /profile e riceve HTML "t9_profile". 5. Il Browser richiede i dati profilo con GET /api/profiles/me (header X-Authenticated-UserId). 6. T9 recupera le statistiche da T4 con GET /games/player/{userId}. 7. Il Browser richiede followers/following via T5 con GET /follow/{targetUserId=userId}; T5 interroga T23. 8. Il sistema mostra il profilo completo (profilo, statistiche e informazioni social).
Flussi alternativi	A1. Credenziali errate o input non valido → risposta 401 e visualizzazione errore. A2. Header X-Authenticated-UserId mancante o non valido su /api/profiles/me → 401 e redirect alla pagina di login. A3. T4 non raggiungibile o timeout (504) → fallback statistiche (valori di default). A4. Servizi social non disponibili o timeout (504) → fallback informazioni followers/following non disponibili.

Tabella 2.2: Scenario d'uso: View Profile

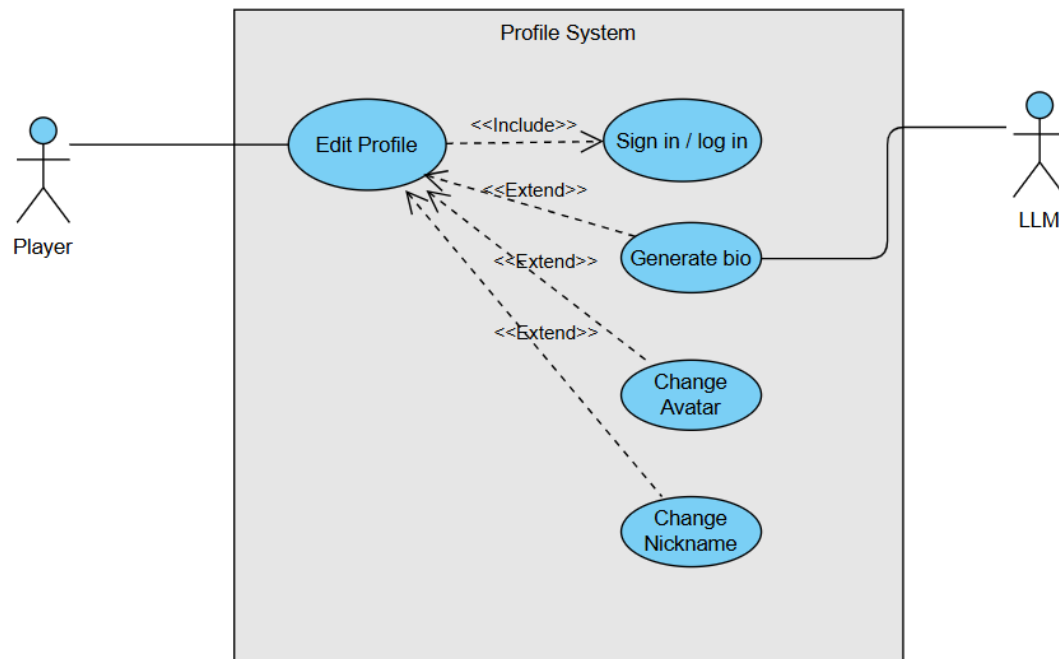


Figura 2.3: Use Case Diagram S2

Campo	Descrizione
ID	UC2
Nome	Edit Profile
Attore primario	Player
Attore secondario	Groq API (LLM)
Descrizione	Il Player modifica il profilo aggiornando nickname, avatar o bio (manuale o generata automaticamente).
Relazione	«include» Sign in / log in «extend» Change Nickname «extend» Change Avatar «extend» Generate bio
Precondizioni	Il Player è autenticato ed è nella schermata di modifica profilo.
Postcondizioni	Le modifiche vengono salvate e T9 restituisce <code>UserProfileResponseDTO</code> .
Flusso principale	1. Il Player modifica un campo del profilo e clicca “Salva”. 2. Il Browser invia la richiesta a T9 con header <code>X-Authenticated-UserId</code>. 3. Per nickname: <code>PUT /api/profiles/me/nickname</code> con body <code>UpdateNicknameRequest</code>. 4. Per avatar: <code>PUT /api/profiles/me/avatar</code> con body <code>UpdateAvatarRequest</code>. 5. Per bio manuale: <code>PUT /api/profiles/me/bio</code> con body <code>UpdateBioRequest</code>. 6. Se OK, T9 risponde 200 OK con <code>UserProfileResponseDTO</code>.
Flussi alternativi	A1. Header mancante o non numerico → 401 Unauthorized (<code>ResponseStatusException</code>). A2. Request non valida (<code>@Valid</code>) → 400 Bad Request con <code>VALIDATION_ERROR</code> (<code>RestExceptionHandler</code>). A3. Nickname già preso → 409 Conflict con <code>NICKNAME_ALREADY_TAKEN</code> (<code>RestExceptionHandler</code>).

Tabella 2.3: Scenario d’uso: Edit Profile

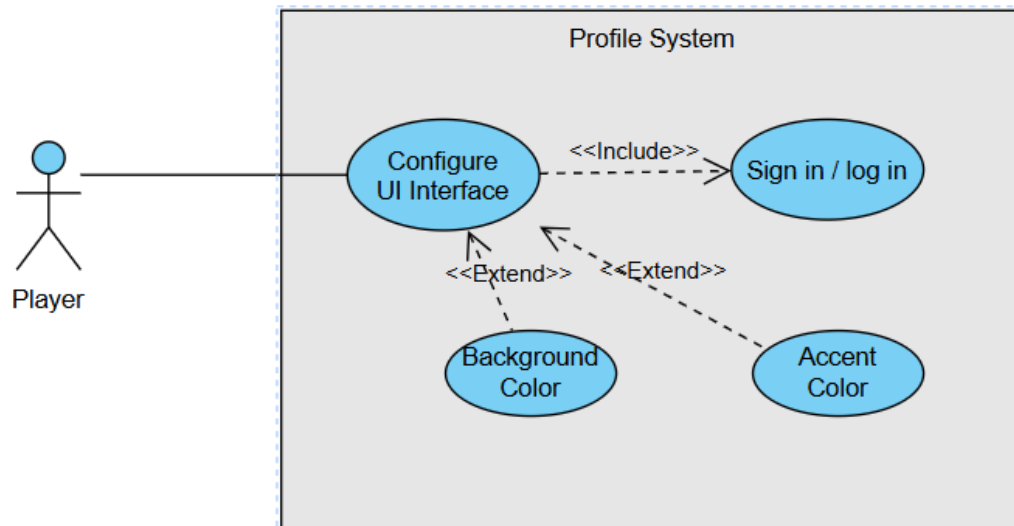


Figura 2.4: Use Case Diagram S3

Campo	Descrizione
ID	UC3
Nome	Configure UI Interface
Attore primario	Player
Attore secondario	T9 UserProfileController
Descrizione	Il Player personalizza l'interfaccia modificando il colore di sfondo (themeColor) e/o il colore di accento (accentColor).
Relazione	«include» Sign in / log in «extend» Background Color «extend» Accent Color
Precondizioni	Il Player è autenticato ed è nella schermata di configurazione dell'interfaccia utente.
Postcondizioni	Le preferenze dell'interfaccia vengono aggiornate e restituite all'interno di UserProfileResponseDTO .
Flusso principale	1. Il Player modifica themeColor e/o accentColor e clicca "Salva". 2. Il Browser invia PUT /api/profiles/me/preferences. Header: X-Authenticated-UserId. Body: UpdatePreferencesRequest (themeColor?, accentColor?, language?, publicProfile?). 3. T9 valida i dati ricevuti e aggiorna le preferenze del profilo. 4. T9 risponde con 200 OK e UserProfileResponseDTO.
Flussi alternativi	A1. Header mancante o vuoto → 401 Unauthorized (ResponseStatusException). A2. Request non valida (@Valid) → 400 Bad Request con VALIDATION_ERROR (RestExceptionHandler).

Tabella 2.4: Scenario d'uso: Configure UI Interface

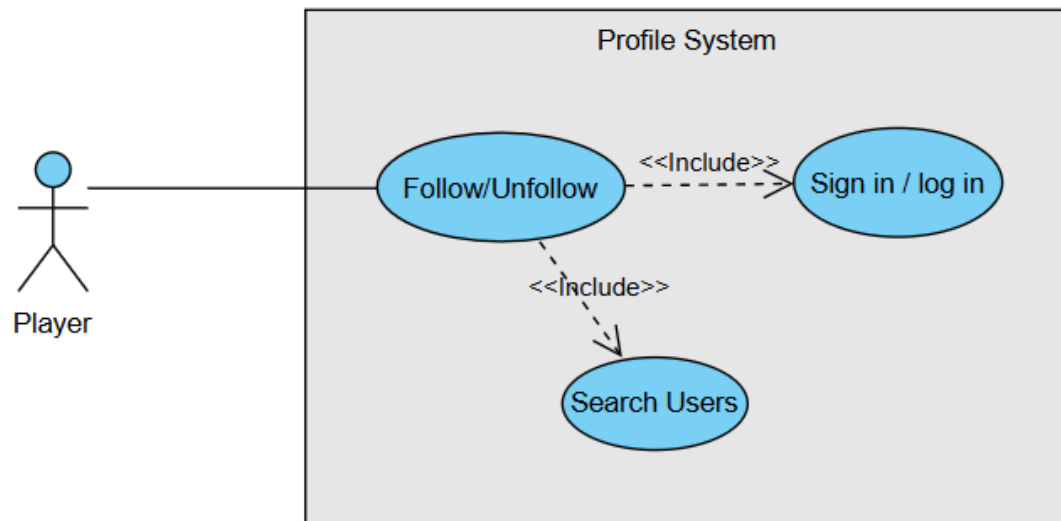


Figura 2.5: Use Case Diagram S4

Campo	Descrizione
ID	UC4
Nome	Follow/Unfollow
Attore primario	Player
Attore secondario	–
Descrizione	Il Player cerca un profilo e può eseguire il toggle follow/unfollow su un utente target, con aggiornamento dello stato e dei contatori quando disponibili.
Relazione	«include» Sign in / log in «include» Search Users
Precondizioni	Il Player è nella schermata di ricerca; per il toggle follow/unfollow è necessario un JWT valido (cookie).
Postcondizioni	Lo stato di follow viene aggiornato e i contatori restituiti riflettono il nuovo stato (quando disponibili).
Flusso principale	1. Il Player inserisce un <code>searchTerm</code>. 2. Il Browser invia <code>GET /api/profiles/search?searchTerm=...&page=...&size=...</code> (header <code>X-Authenticated-UserId</code>). 3. T9 risponde con 200 OK e <code>Page<UserProfileSearchDTO></code>. 4. Il Player seleziona un utente target e il Browser invia <code>GET /follow/{targetUserId}</code>. 5. T5 interroga T23 con <code>GET /profile/followers?userId={targetUserId}</code> e <code>GET /profile/following?userId={targetUserId}</code>. 6. Se il JWT è valido, T5 interroga anche <code>GET /profile/isFollowing?followerId={authUserId}&followingId={targetUserId}</code>. 7. Il Player clicca follow/unfollow e il Browser invia <code>POST /follow/{targetUserId}</code> (cookie JWT). 8. T5 invoca T23 con <code>POST /profile/toggle_follow?followerId={authUserId}&followingId={targetUserId}</code>. 9. T5 aggiorna i contatori richiamando i servizi followers/following e restituisce i nuovi valori al Browser.
Flussi alternativi	- A1. Ricerca: header mancante o non valido → 401 Unauthorized (<code>ResponseStatusException</code>). - A2. Ricerca: <code>searchTerm</code> vuoto o blank → 200 OK con <code>Page.empty(pageable)</code>. - A3. <code>GET /follow/{targetUserId}</code> con <code>targetUserId</code> nullo → 400 Bad Request (errore: “Missing targetUserId”). - A4. Toggle: JWT mancante o non valido → 401 Unauthorized (errore: “Not authenticated”). - A5. Risposte da T23: utente non trovato → 400 Bad Request; errore imprevisto → 500 Internal Server Error.

Tabella 2.5: Scenario d’uso: Follow/Unfollow

Capitolo 3

Progettazione della Soluzione

3.1 View dei Componenti e Interfaccia REST API

3.1.1 Scelte architetturali

Per soddisfare i requisiti funzionali e non funzionali emersi nella fase di analisi, è stata introdotta una nuova componente architetturale dedicata alla gestione del profilo utente, denominata **T9 Profile Service**.

La scelta di un microservizio separato è motivata dai principi dell'architettura a microservizi, in particolare:

- separazione delle responsabilità;
- modularità;
- indipendenza evolutiva dei componenti.

3.1.2 Vista a Componenti e Interazioni tra i Servizi

Per chiarire in modo più dettagliato la struttura interna del sistema e le responsabilità dei singoli microservizi, oltre alla vista C&C è stata modellata una **vista a componenti**, riportata in Figura 3.1. Questa vista consente di esplicitare i componenti software principali, le interfacce REST esposte e le modalità di interazione tra i servizi, fornendo una rappresentazione più fine rispetto alla sola struttura di alto livello.

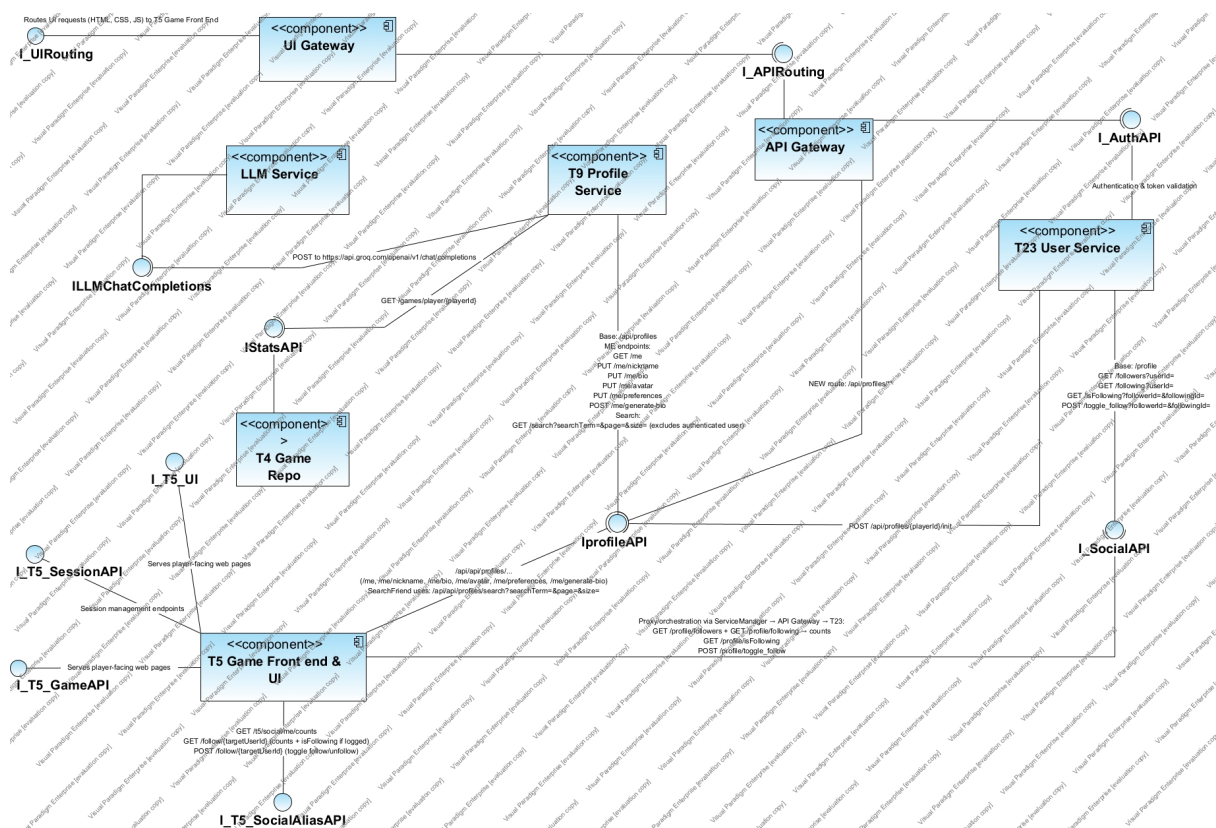


Figura 3.1: Component Diagram dell'architettura TO-BE

Nel diagramma sono evidenziati i principali componenti dell'architettura e le rispettive responsabilità. Il **T5 Game Front End / UI** rappresenta il punto di accesso per l'utente finale e gestisce le interazioni tramite richieste HTTP. Tutte le richieste provenienti dal frontend vengono indirizzate verso l'**API Gateway**, che funge da punto di ingresso unico al sistema.

L'**API Gateway** svolge un ruolo centrale di orchestrazione e routing, occupandosi dell'instradamento delle richieste verso i microservizi appropriati e dell'applicazione dei meccanismi di autenticazione e autorizzazione, delegati al microservizio **T23 User Service**. Quest'ultimo espone interfacce REST dedicate esclusivamente alla validazione dei token e alla gestione delle credenziali, senza partecipare direttamente alle logiche di dominio del profilo utente.

Il **T9 Profile Service**, introdotto come parte dell'evoluzione architetturale, espone una propria **REST API** che costituisce l'unico punto di accesso ai dati di profilo del giocatore. Come mostrato nel diagramma, il servizio viene invocato tramite l'API Gateway attraverso chiamate REST di tipo **GET** e **POST**. In particolare:

- le operazioni di recupero del profilo sono realizzate tramite chiamate GET (ad esempio `/api/profiles/me` e `/api/profiles/{userId}`);
- le funzionalità di ricerca e inizializzazione del profilo sono implementate tramite chiamate POST (ad esempio `/api/profiles/search`);

- la generazione automatica della biografia del profilo avviene tramite una chiamata `POST /api/profiles/me/generate-bio`.

Dal punto di vista delle dipendenze *service-to-service*, il microservizio T9 interagisce con il **T4 Game Repository** per l'accesso alle informazioni di gioco del giocatore, tramite una chiamata REST di tipo `GET /games/player/{playerId}`. Questa interazione consente di arricchire il profilo utente con dati relativi ai progressi di gioco, mantenendo al contempo una chiara separazione tra i domini applicativi.

Il diagramma mette inoltre in evidenza l'integrazione di un **servizio esterno di Intelligenza Artificiale (LLM Service)**, utilizzato dal microservizio T9 per la generazione automatica delle biografie. Tale integrazione avviene tramite una chiamata `POST` verso un'API esterna compatibile con OpenAI (`/chat/completions`), rappresentando un'estensione funzionale significativa rispetto all'architettura originale.

Nel complesso, la vista a componenti evidenzia come l'introduzione del microservizio T9 consenta una chiara separazione delle responsabilità, una migliore modularità del sistema e un'integrazione controllata delle nuove funzionalità, mantenendo la coerenza con l'architettura a microservizi già esistente.

3.1.3 Interfaccia REST del microservizio T9

Il microservizio T9 espone un insieme di API REST dedicate alla gestione del profilo utente. Le principali funzionalità offerte includono:

- recupero del profilo dell'utente autenticato;
- aggiornamento delle informazioni del profilo;
- aggiornamenti puntuali (nickname, bio, avatar, preferenze);
- generazione automatica della biografia tramite un servizio LLM esterno;
- consultazione e ricerca di profili utente.

Le API sono accessibili esclusivamente tramite l'API Gateway e seguono i principi RESTful. L'identità dell'utente autenticato viene propagata tramite header HTTP, garantendo sicurezza e coerenza nell'accesso ai dati.

3.1.4 Ricerca del profilo utente e integrazione della componente social

La ricerca dei profili utente è realizzata tramite endpoint REST distribuiti su più microservizi, ciascuno con responsabilità ben definite.

In particolare:

- il microservizio **T9 Profile Service** gestisce la ricerca dei profili dal punto di vista informativo, consentendo di individuare utenti sulla base di attributi quali nickname o nome;

- il microservizio **T23 User Service** gestisce la componente social, includendo le relazioni di follow/unfollow e il calcolo dei conteggi di follower e following.

Quando un utente ricerca il profilo di un altro giocatore (ad esempio un amico), la richiesta viene instradata tramite l'API Gateway verso T9, che restituisce i profili corrispondenti ai criteri di ricerca. Per ciascun profilo individuato, le informazioni sociali vengono recuperate interrogando T23.

Dal punto di vista architetturale, la vista completa del profilo è ottenuta tramite aggregazione a livello applicativo, senza accessi diretti ai database tra microservizi.

3.1.5 Integrazione tramite API Gateway

Tutte le richieste provenienti dal frontend sono gestite dall'API Gateway, che si occupa del routing verso T9, T23 e gli altri microservizi. Per vincoli di routing dell'infrastruttura, alcune funzionalità social sono esposte tramite endpoint alias nel microservizio **T5**, che funge da punto di appoggio per la comunicazione con T23.

3.2 View Modulare

3.2.1 Package Diagram del microservizio T9

Il Package Diagram descrive l'organizzazione interna del microservizio T9, evidenziando la suddivisione in moduli coerenti e le dipendenze tra di essi.

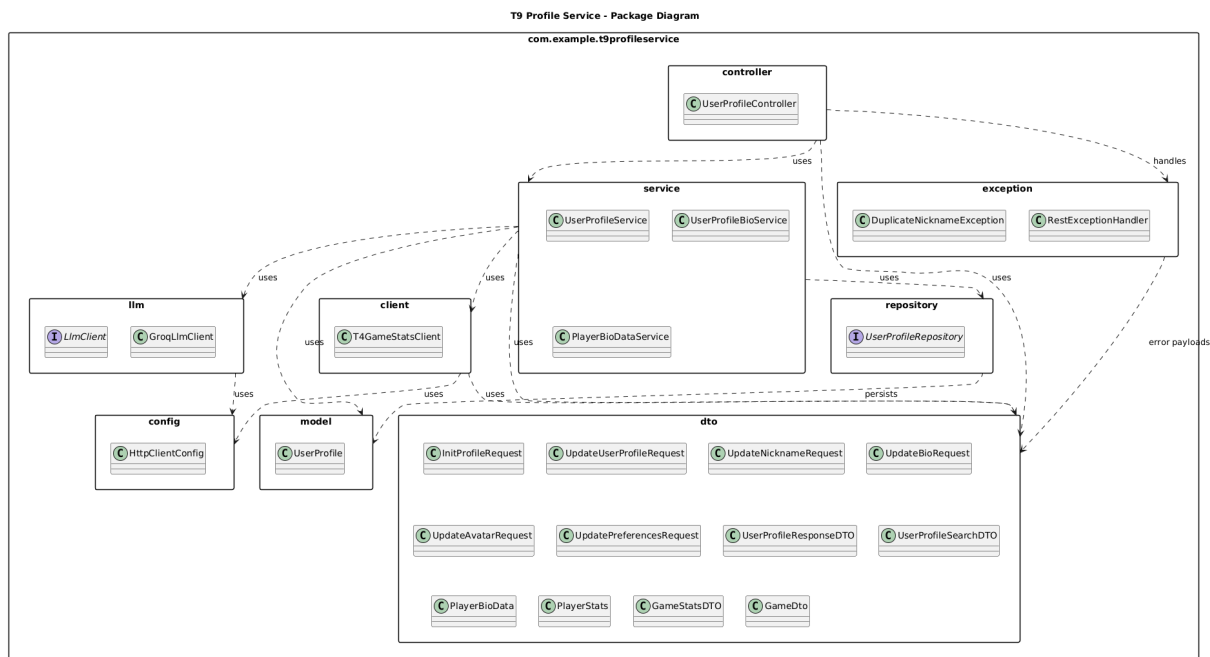


Figura 3.2: Package Diagram del microservizio T9

Il microservizio è organizzato secondo una struttura a livelli:

- **controller**: esposizione degli endpoint REST;
- **service**: logica applicativa e orchestrazione delle operazioni;
- **repository**: accesso al database;
- **model**: entità di dominio persistenti;
- **dto**: oggetti di trasferimento dati;
- **client** e **llm**: integrazione con servizi esterni;
- **exception**: gestione centralizzata degli errori.

Questa organizzazione favorisce manutenibilità, testabilità ed estendibilità del micro-servizio.

3.3 View Dinamica

3.3.1 Sequence Diagram

La view dinamica descrive il comportamento del sistema nei principali casi d'uso relativi alla gestione del profilo. I diagrammi di sequenza modellano:

- S1: recupero del profilo dell'utente autenticato;
- S2: modifica informazioni profilo utente (biografia, avatar, nickname);
- S3: configurazione Interfaccia Utente (colore background e accent);
- S4: Consultazione lista utenti, ricerca e seguire utente.

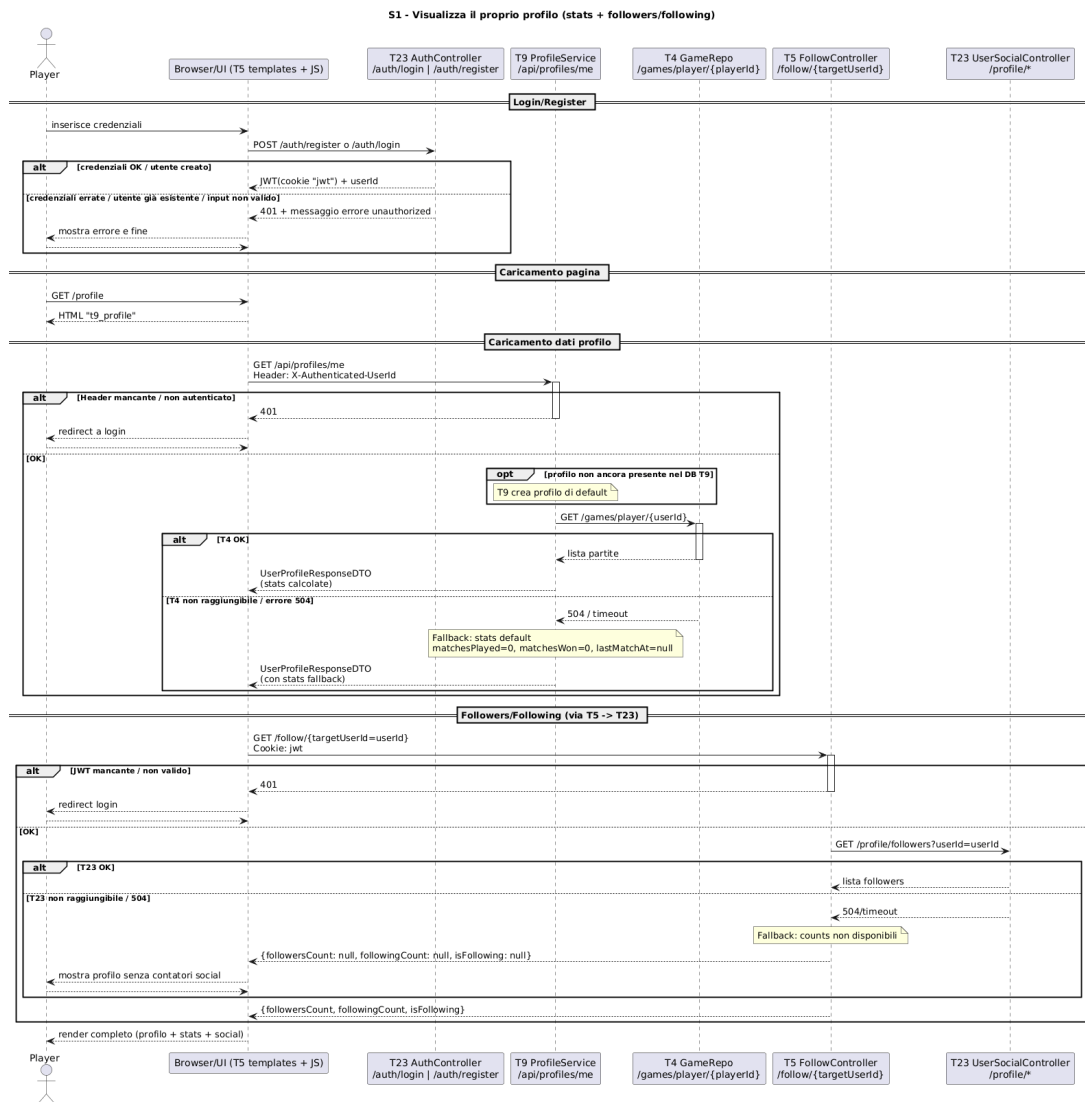


Figura 3.3: Sequence Diagram S1

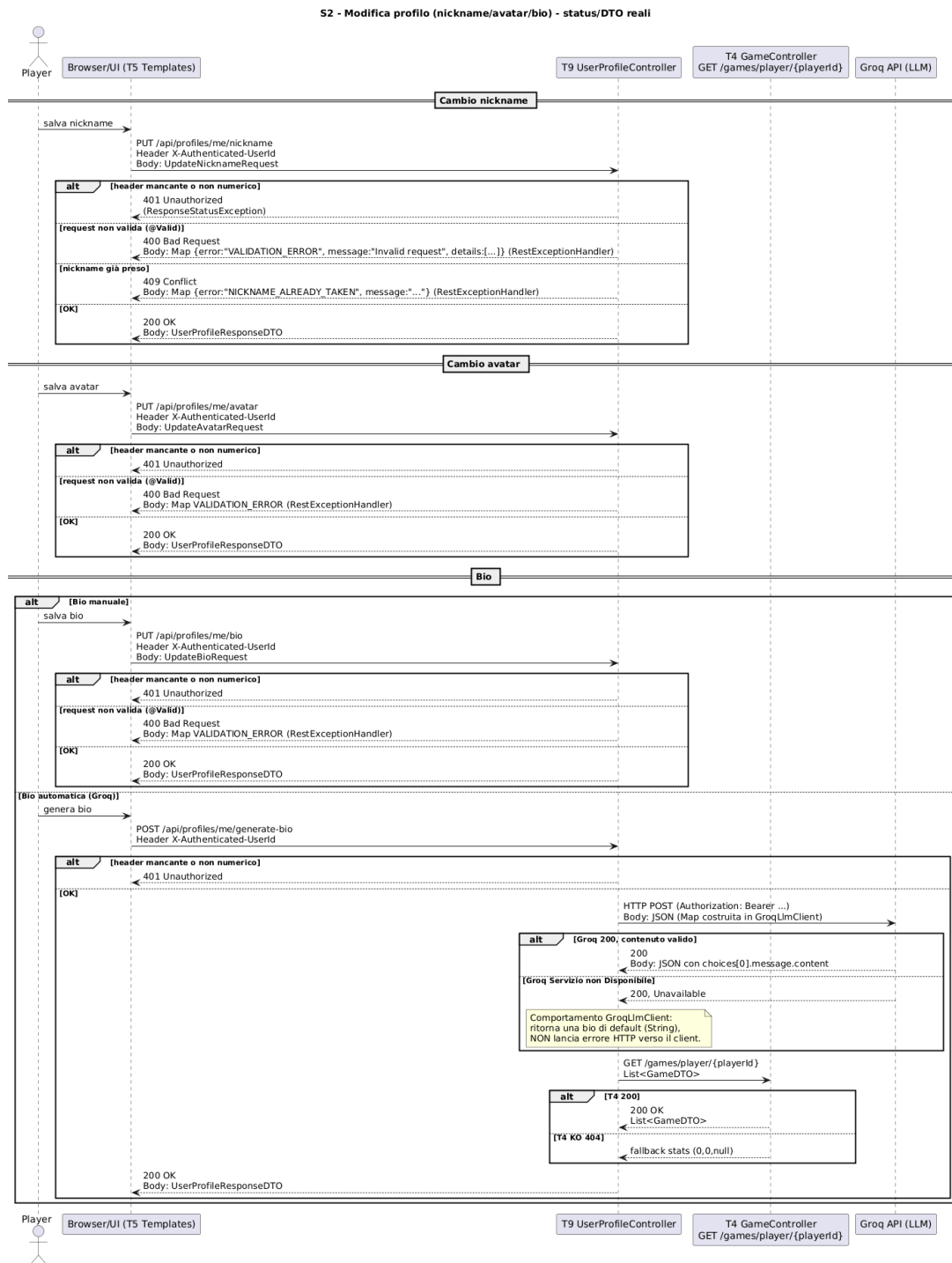


Figura 3.4: Sequence Diagram S2

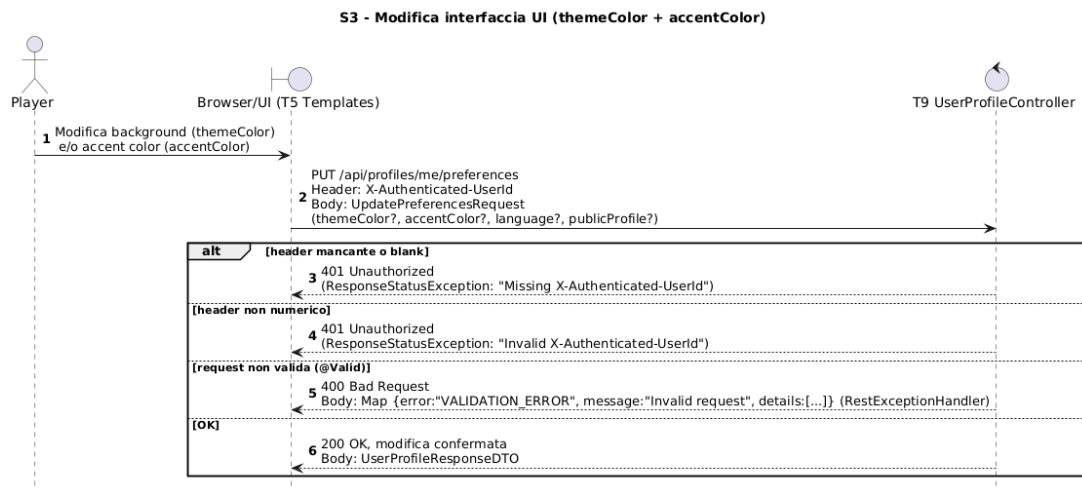


Figura 3.5: Sequence Diagram S3

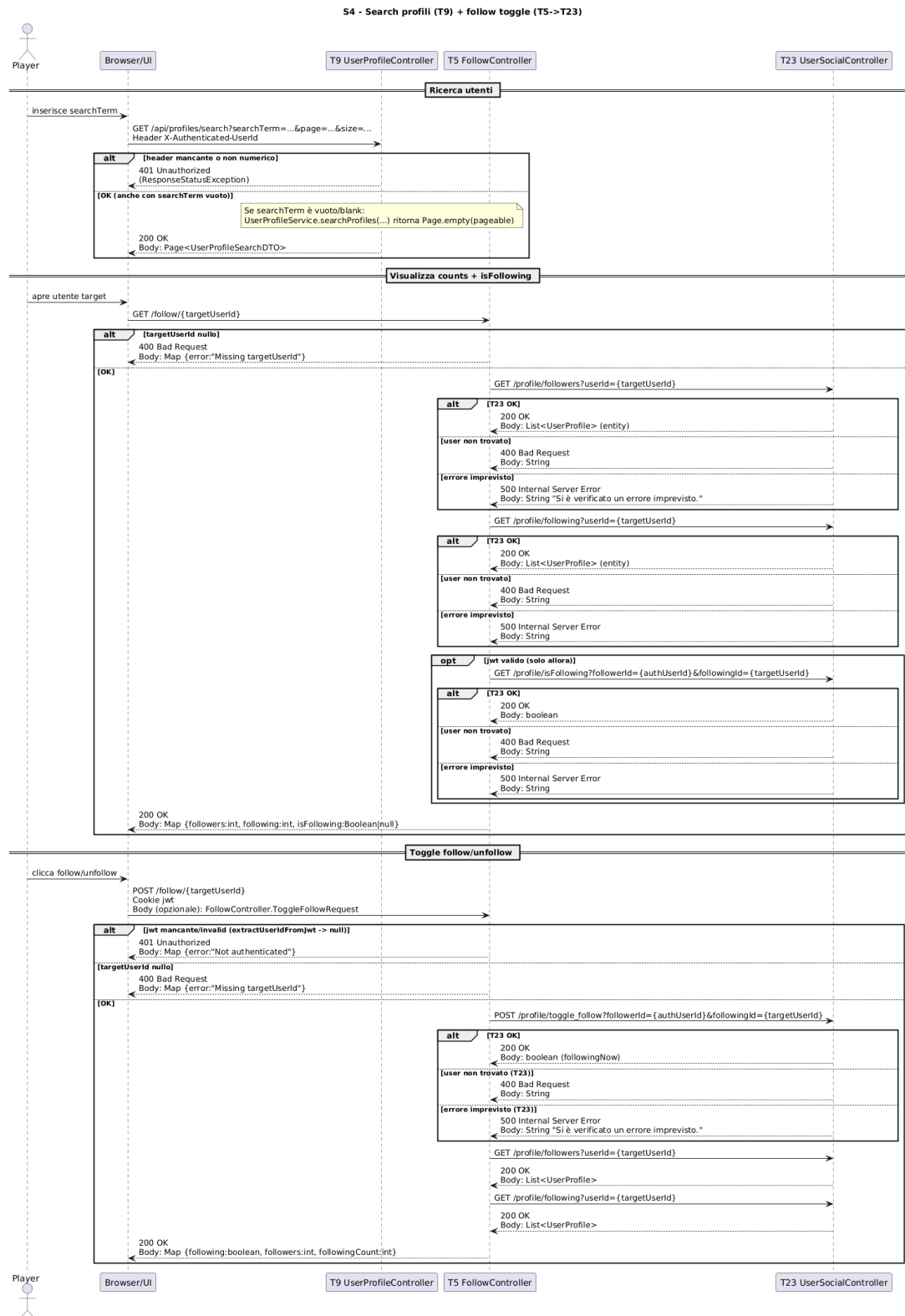


Figura 3.6: Sequence Diagram S4

In tutti i casi, l'API Gateway agisce come punto di orchestrazione delle chiamate REST tra frontend e microservizi backend.

3.4 Prototipo dell'Interfaccia Utente

3.4.1 Obiettivi di progettazione della UI

La progettazione dell'interfaccia utente è stata orientata alla creazione di una **pagina di profilo unificata**, in grado di presentare in modo coerente informazioni provenienti da più microservizi.

Gli obiettivi principali sono stati:

- mostrare in modo chiaro le informazioni di profilo;
- consentire la modifica immediata dei dati personali;
- consentire la modifica dell'interfaccia grafica dell'utente;
- supportare la ricerca, la consultazione e la possibilità di seguire i profili di altri utenti.

3.4.2 Activity Diagram

Il diagramma delle attività rappresenta il flusso di controllo delle operazioni eseguite durante un processo del sistema, mettendo in evidenza le diverse attività, le decisioni e i possibili percorsi alternativi. In questo caso, il diagramma viene utilizzato per descrivere in modo chiaro il comportamento del sistema durante operazioni che coinvolgono più passaggi logici.

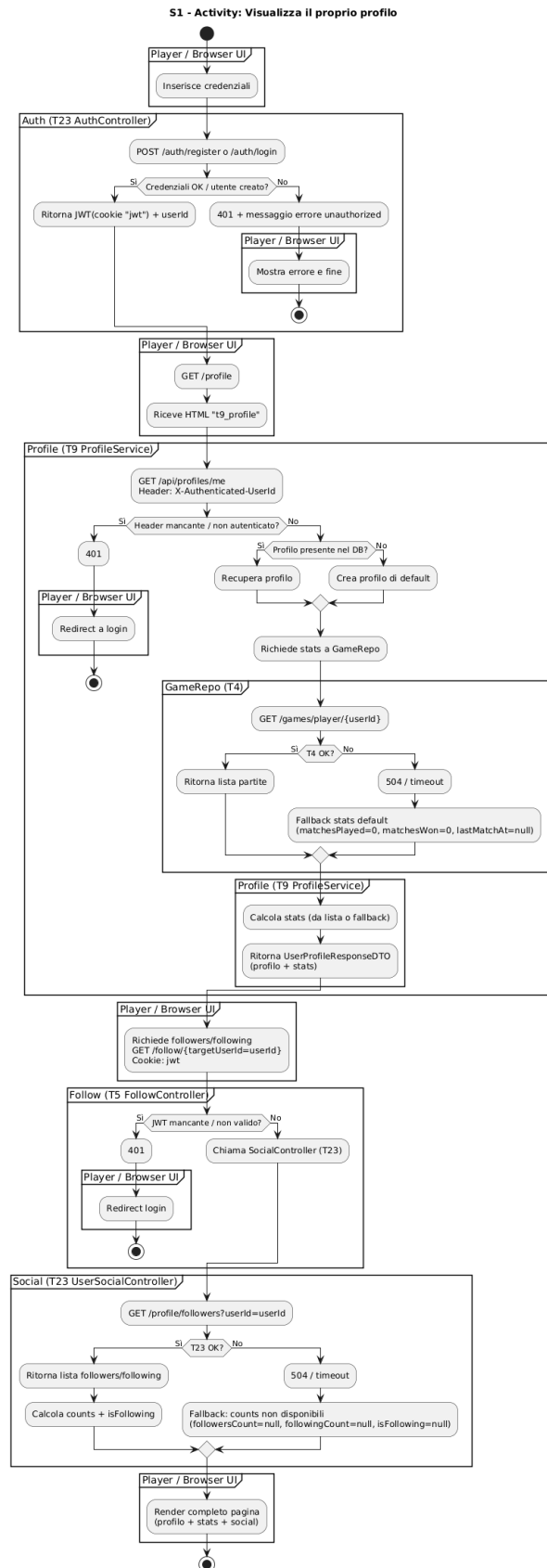


Figura 3.7: Activity Diagram della Visualizzazione del profilo

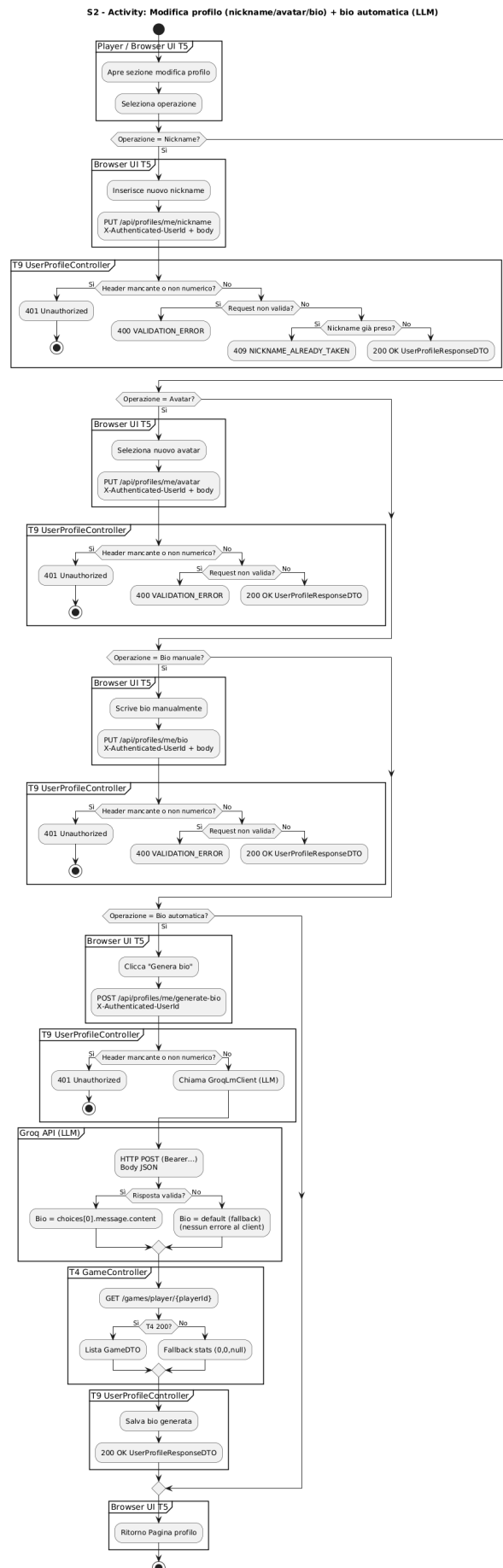


Figura 3.8: Activity Diagram della Modifica del profilo

Activity Diagram - S3 Modifica interfaccia UI (themeColor + accentColor) - T9 reale

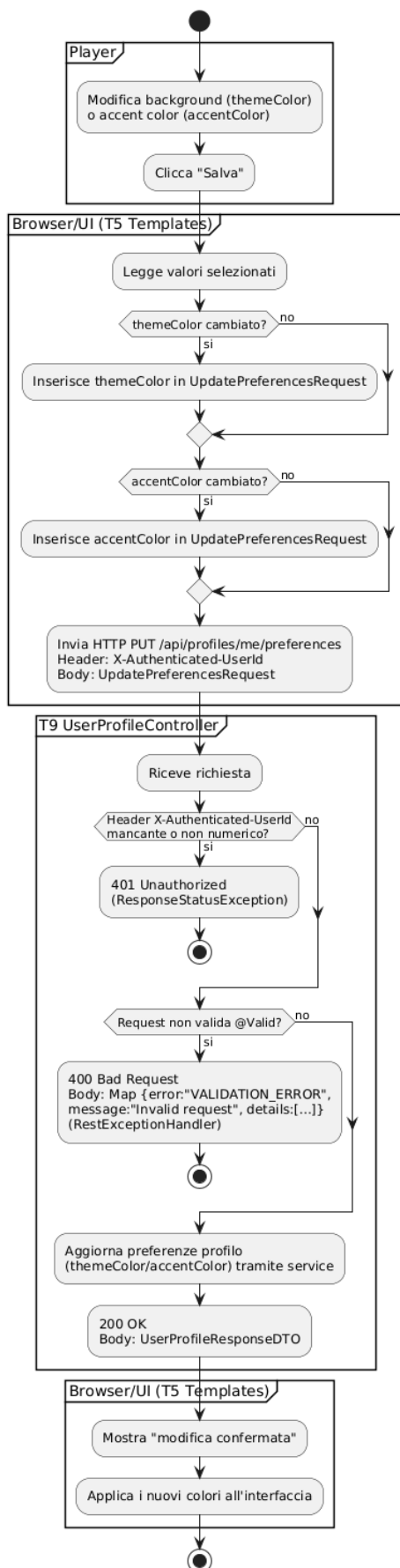


Figura 3.9: Activity Diagram della Modifica UI del profilo

Activity Diagram - S3 Modifica interfaccia UI (themeColor / accentColor)

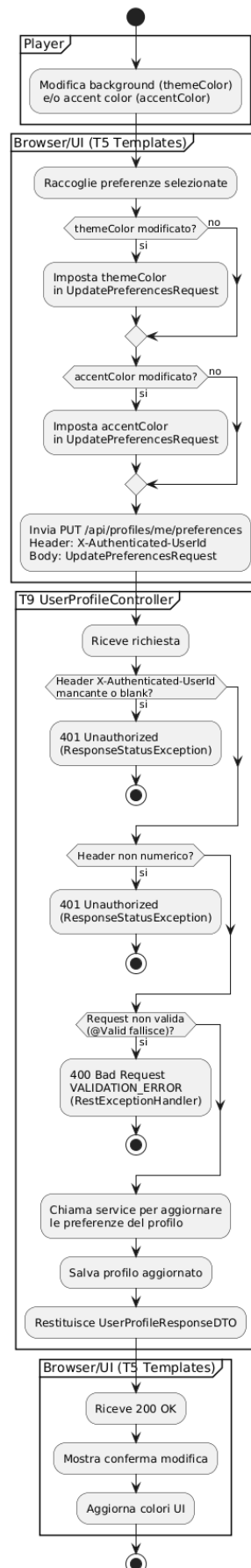


Figura 3.10: Activity Diagram dell'interazione Social dell'utente

3.4.3 Prototipo UI

Questa sezione presenta il prototipo dell'interfaccia utente del sistema di gestione del profilo, mostrando le principali funzionalità disponibili per il Player. Le schermate illustrano i casi d'uso descritti nei capitoli precedenti, in particolare la visualizzazione del profilo, la modifica delle informazioni personali, la personalizzazione dell'interfaccia e l'interazione sociale tra utenti.

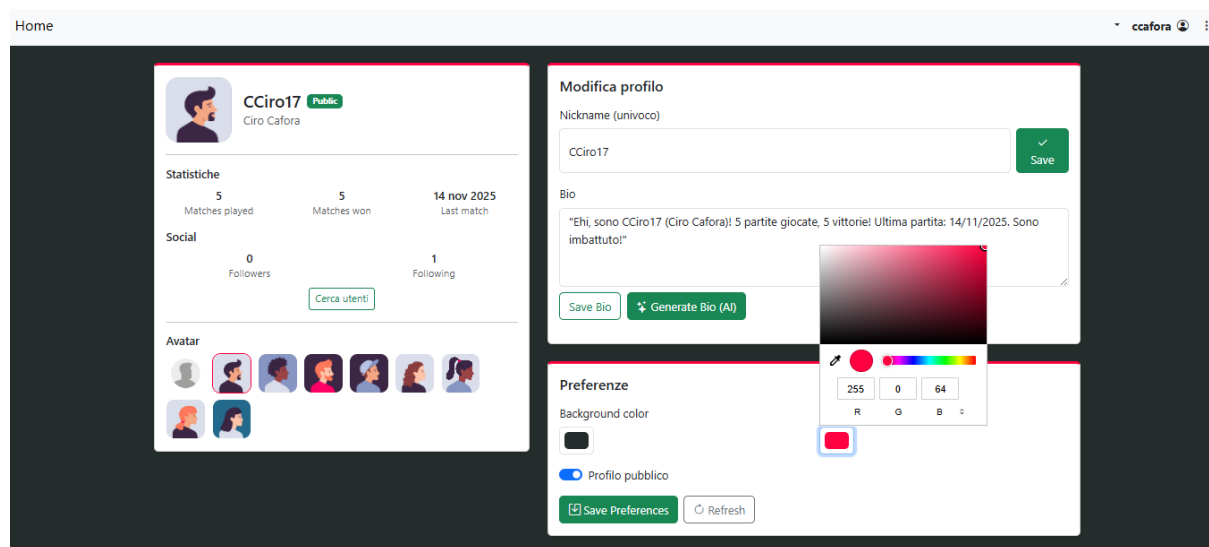


Figura 3.11: Schermata di modifica del profilo e personalizzazione dell'interfaccia

La Figura 3.11 mostra la schermata principale di modifica del profilo. Sulla sinistra è visibile il riepilogo del profilo del Player, comprensivo di avatar selezionato, statistiche di gioco (partite giocate, vittorie, ultima partita) e informazioni sociali (followers e following). Sulla destra sono presenti i pannelli di modifica del profilo, che consentono di aggiornare il nickname, la biografia (con possibilità di generazione automatica tramite AI) e le preferenze dell'interfaccia, come il colore di sfondo e il colore di accento.



Figura 3.12: Gestione errore di nickname già in uso

La Figura 3.12 mostra la gestione di un errore durante la modifica del nickname. Nel caso in cui il Player inserisca un nickname già esistente nel sistema, l'interfaccia visualizza

un messaggio di errore esplicativo, invitando l'utente a sceglierne uno differente. Questo comportamento riflette il controllo di unicità gestito dal backend e descritto nel caso d'uso di modifica profilo.

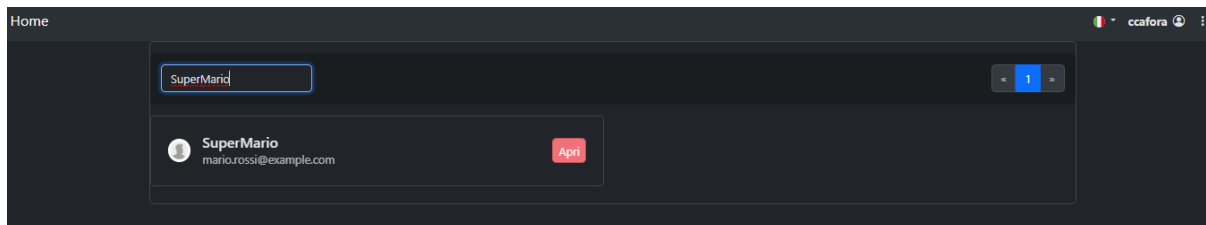


Figura 3.13: Risultati della ricerca utenti

La Figura 3.13 rappresenta la schermata di ricerca degli utenti. Il Player può inserire un termine di ricerca per individuare altri profili registrati nel sistema. Per ciascun risultato vengono mostrati nickname ed email, consentendo all'utente di selezionare il profilo di interesse e accedere alla sua pagina pubblica.

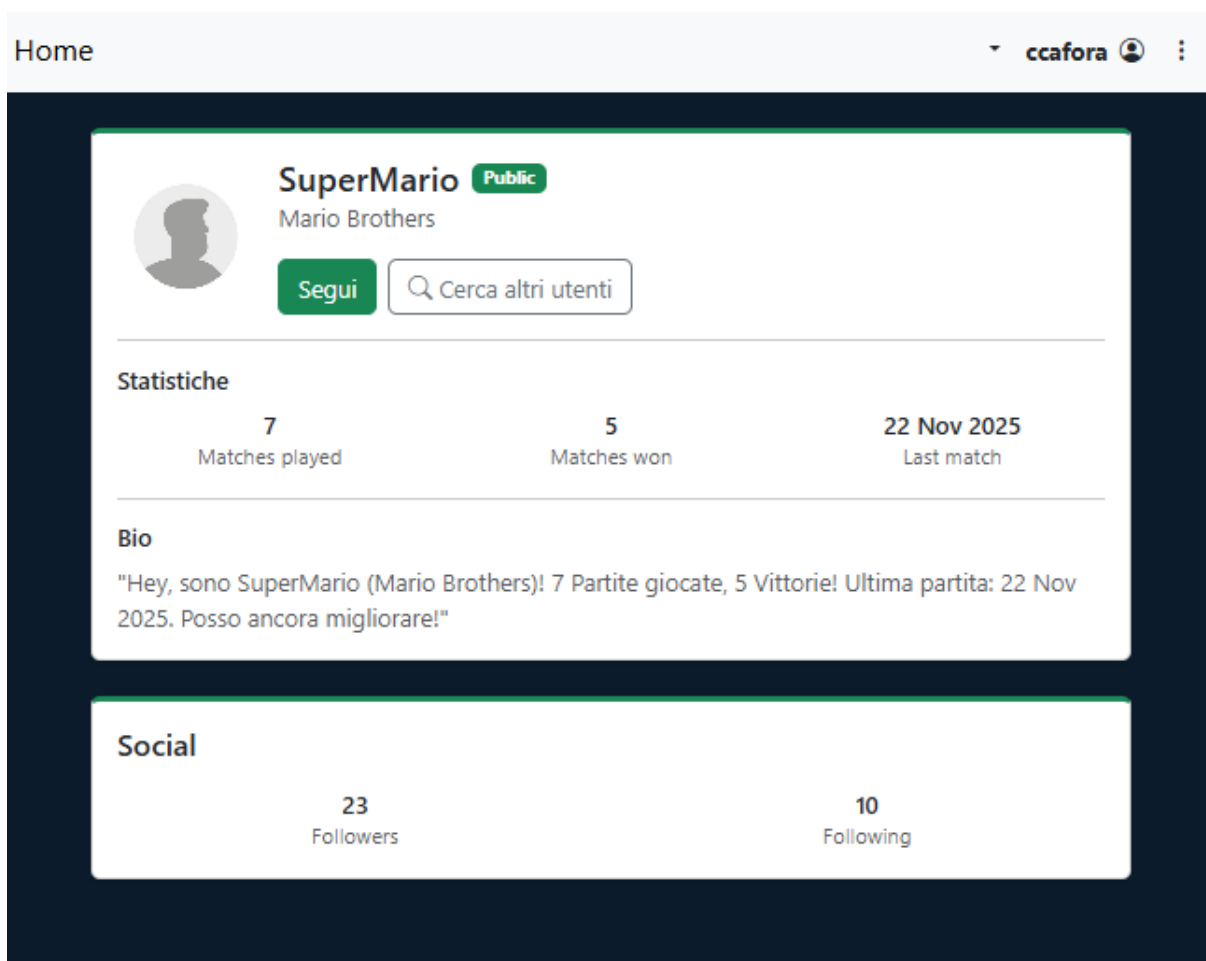


Figura 3.14: Visualizzazione profilo pubblico di un altro utente

La Figura 3.14 mostra la visualizzazione del profilo pubblico di un altro Player. In questa schermata sono visibili le statistiche di gioco, la biografia e i contatori sociali. È inoltre presente il pulsante *Segui*, che permette di avviare l'azione di follow verso l'utente selezionato, in accordo con il caso d'uso di follow/unfollow.

3.5 Deployment Diagram dei Microservizi Coinvolti

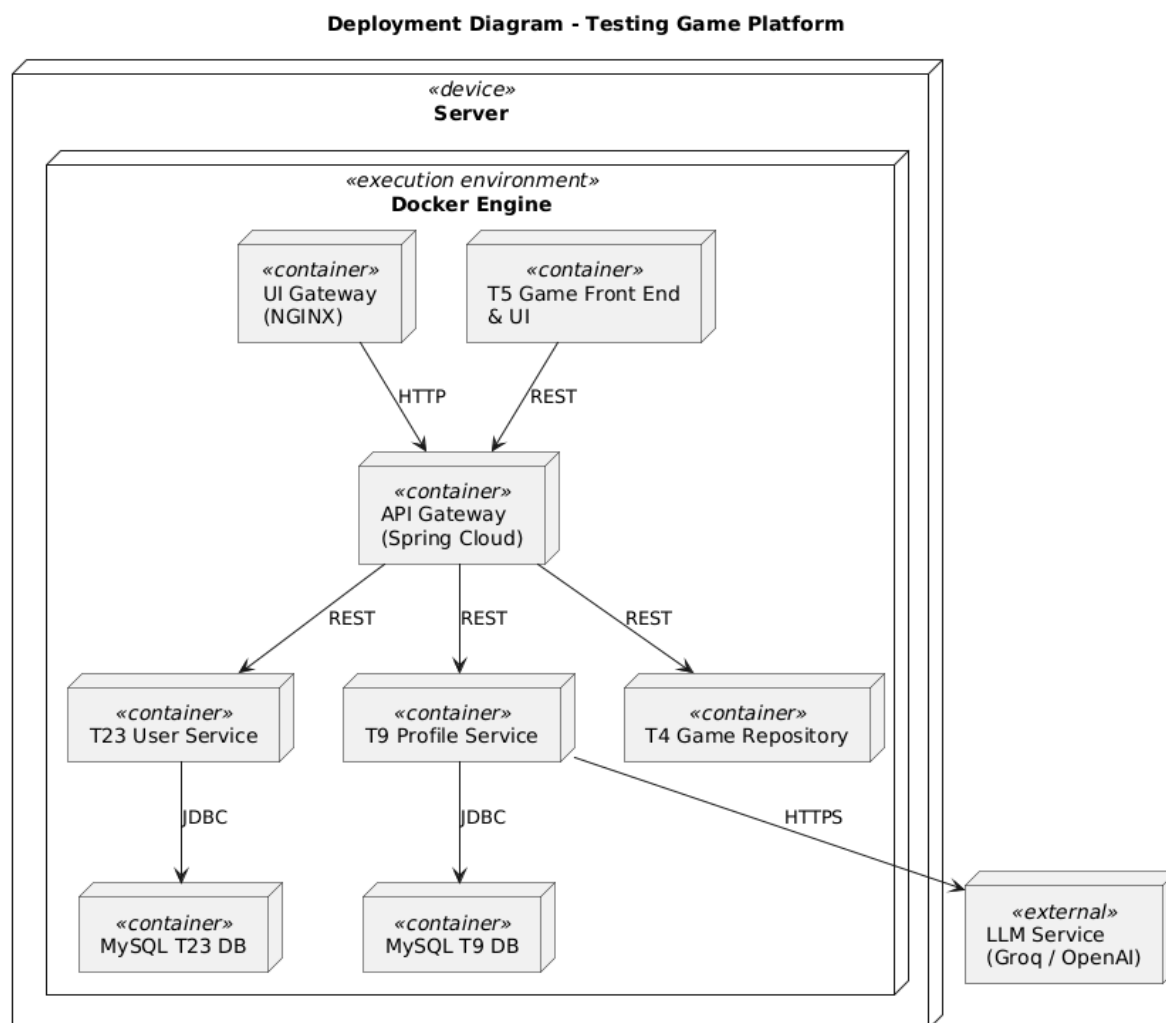


Figura 3.15: Deployment Diagram dei microservizi coinvolti

Il Deployment Diagram rappresenta la distribuzione fisica dei **microservizi direttamente coinvolti nell'evoluzione architetturale introdotta**, con particolare riferimento al nuovo microservizio **T9 (Profile Service)** e ai componenti che interagiscono con esso.

L'ambiente di esecuzione è basato su container Docker: ciascun microservizio è distribuito all'interno di un container dedicato ed è progettato per operare in modo indipendente rispetto agli altri. In particolare, il microservizio T9 dispone di un **database**

autonomo, garantendo isolamento dei dati e indipendenza evolutiva rispetto ai servizi legacy.

I microservizi non direttamente interessati dalle modifiche architetturali non sono riportati nel diagramma, al fine di mantenere una rappresentazione focalizzata sugli elementi rilevanti per il task assegnato.

3.6 Progettazione del Database

3.6.1 Database del microservizio T9

Il microservizio T9 utilizza un database dedicato per la persistenza delle informazioni di profilo dell'utente. Non sono memorizzate informazioni relative all'autenticazione o alle relazioni sociali.

3.6.2 Modello Entity-Relationship

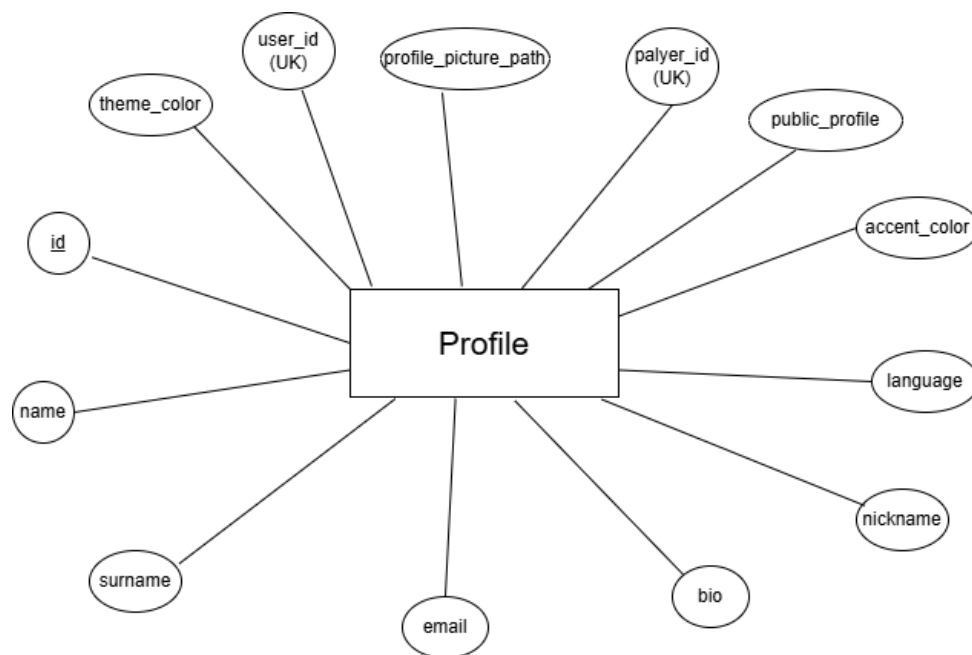


Figura 3.16: Modello E/R del database del microservizio T9

3.6.3 Struttura della tabella Profile

Tabella 3.1: Struttura della tabella profile

Campo	Tipo	Descrizione
id	Long	Identificativo univoco del profilo.
user_id	Long	Identificativo logico dell'utente.
player_id	Long	Identificativo del giocatore.
name	String	Nome dell'utente.
surname	String	Cognome dell'utente.
email	String	Email dell'utente.
nickname	String	Nickname pubblico.
bio	String	Biografia del profilo.
profile_picture_path	String	Percorso avatar.
theme_color	String	Colore principale UI.
accent_color	String	Colore secondario UI.
language	String	Lingua preferita.
public_profile	Boolean	Visibilità del profilo.

3.6.4 Ruolo del microservizio T23 nella gestione social

Le funzionalità social (follow, unfollow, conteggio dei follower e following) sono di esclusiva competenza del microservizio **T23**. T9 non memorizza alcuna relazione sociale.

La vista completa del profilo è ottenuta aggregando a livello applicativo:

- dati di profilo da T9;
- dati social da T23.

Questo approccio consente di mantenere basso accoppiamento tra microservizi e coerenza architetturale.

3.7 Issue e Anti-pattern

Durante lo sviluppo del progetto sono state individuate alcune criticità architetturali e organizzative, emerse principalmente a causa dell'evoluzione incrementale del sistema e della presenza di funzionalità parzialmente implementate da precedenti interventi.

L'analisi di tali problematiche ha permesso di individuare potenziali anti-pattern e di adottare soluzioni progettuali mirate, in linea con i principi dell'architettura a microservizi, migliorando la manutenibilità, la modularità e l'estendibilità complessiva del sistema.

Tabella 3.2: Issue e anti-pattern rilevati durante lo sviluppo

Issue Rilevata	Anti-pattern Associato	Soluzione Adottata
Frammentazione delle informazioni del profilo utente tra più microservizi e accesso tramite elementi di interfaccia non coerenti	Scattered Responsibilities	Introduzione del microservizio T9 come componente dedicato alla gestione del profilo utente, centralizzando i dati di profilo e migliorandone la fruizione.
Tentativo iniziale di sviluppo su una copia del microservizio T23 con problemi di sincronizzazione del database e gestione dell'autenticazione	Tight Coupling tra servizi	Abbandono dell'approccio iniziale e adozione di un'integrazione basata su API REST ben definite, riducendo l'accoppiamento tra i microservizi.
Presenza di codice UI incompleto o non utilizzato relativo a una precedente progettazione della pagina di profilo	Dead Code / Incomplete Feature	Recupero e adattamento delle parti riutilizzabili del codice, integrandole correttamente nella nuova architettura basata su T9.
Rischio di accesso diretto o condivisione del database tra microservizi	Shared Database	Adozione di database dedicati per ciascun microservizio e sincronizzazione dei dati esclusivamente tramite comunicazione applicativa.
Dipendenza potenziale tra registrazione utente e disponibilità del servizio di profilo	Distributed Transaction	Progettazione dell'inizializzazione del profilo come operazione <i>best-effort</i> , evitando transazioni distribuite e migliorando la resilienza del sistema.

La risoluzione delle issue individuate ha contribuito a rafforzare la coerenza architetturale, riducendo l'accoppiamento tra i componenti e favorendo l'evoluzione del progetto.

Capitolo 4

Implementazione e Struttura del Progetto

Il presente capitolo descrive le principali attività di implementazione svolte per realizzare la soluzione progettata, con particolare attenzione alle modifiche apportate ai microservizi esistenti e alla creazione del nuovo microservizio T9.

L'implementazione è stata condotta nel rispetto delle decisioni architetturali descritte nel Capitolo 3, mantenendo una chiara separazione delle responsabilità tra i microservizi e limitando l'impatto sui componenti preesistenti. Le modifiche hanno interessato selettivamente i microservizi T23 e T5, mentre il microservizio T9 è stato sviluppato interamente da zero.

Ulteriori interventi hanno riguardato esclusivamente la configurazione dell'API Gateway per abilitare il routing verso il nuovo servizio.

Nel seguito vengono riportati, in forma strutturata, i file modificati o creati per ciascun microservizio, seguiti da un'analisi di dettaglio delle componenti implementative più rilevanti.

4.1 Elenco Moduli Aggiunti o Modificati

Tabella 4.1: Riepilogo dei file modificati o creati per microservizio

Microservizio	Percorso	File Modificati / Creati
T23	src/main/example/ db_setup	Readme.md, docker-compose.yml, pom.xml config/TestTemplateConfig.java controller/PlayerController.java mapper/MapperFacade.java model/dto/gamification/PlayerProgressDTO.java model/repository/UserProfileRepository.java security/WebSecurityConfig.java service/AuthService.java, PlayerService.java, UserSocialService.java
T23	src/main/resources	application.properties, application-prod.properties language/message_*.properties static/t23/js/home.js, login_admin.js templates/home.html, Login_admin.html
T5	src/main/java/com/ G2/controllers	UserProfileController.java SocialGraphController.java SocialGraphAliasController.java FollowController.java GuiController.java
T5	src/main/resources/ static/T5/js	profile.js friend_profile.js util_editor.js
T5	src/main/resources/ templates	t9_profile.html friend_profile.html search.html Leaderboard.html fragments/Navbar.html

Continua nella pagina seguente...

Microservizio	Percorso	File Modificati / Creati (continua)
T5	root / config / deploy	Dockerfile docker-compose.yml pom.xml application.properties logback-spring.xml
T9	Intero microservizio	Microservizio sviluppato interamente da zero (controller, service, repository, model, DTO, configurazioni, Docker, database).
API Gateway	src/main/resources	application.yml (modifica configurativa per routing verso T9).

4.2 Close-up sui File Principali

4.2.1 T9 – UserProfileBioService.java

Porzione analizzata La porzione mostra la pipeline completa:

- costruzione delle informazioni utili(`PlayerBioData`),
- generazione della bio via interfaccia `LlmClient` (iniezione di dipendenza),
- salvataggio persistente nel profilo. Questa separazione consente testabilità e sostituzione del provider LLM.

```

1 public UserProfile generateAndSaveBio(Long userId) {
2     PlayerBioData data = playerBioDataService.buildPlayerBioData(userId);
3     String bio = llmClient.generateBio(data);
4     UserProfile profile = userProfileRepository.findById(userId)
5         .or(() -> userProfileRepository.findByIdByPlayerId(userId))
6         .orElseThrow(() -> new RuntimeException("Profile not found for
            userId/playerId: " + userId));
7
8     profile.setBio(bio);
9     UserProfile saved = userProfileRepository.save(profile);
10    logger.info("Updated bio for user {} (length={})", userId, bio != null ?
        bio.length() : 0);
11    return saved;
12 }

```

Listing 4.1: T9 - Pipeline: build data, generate bio, save su DB (UserProfileBioService)

4.2.2 T9 – PlayerBioDataService.java

Porzione analizzata L'estratto evidenzia l'arricchimento dati: oltre al profilo locale (T9), il servizio tenta di recuperare statistiche da T4. In caso di errore, viene applicato un fallback controllato, evitando che l'assenza di T4 blocchi la generazione della bio.

```
1 public PlayerBioData buildPlayerBioData(Long userId) {
2     UserProfile profile = userProfileRepository.findByUserId(userId).orElse(
3         null);
4     GameStatsDTO stats = null;
5     try {
6         stats = t4GameStatsClient.getStatsForUser(userId);
7     } catch (Exception e) {
8         logger.warn("Failed to retrieve game stats for user {}: {}", userId, e.
9             toString());
10        stats = null;
11    }
12    Integer matchesPlayed = stats != null ? Integer.valueOf(stats.matchesPlayed
13        ()) : null;
14    Integer matchesWon = stats != null ? Integer.valueOf(stats.matchesWon()) :
15        null;
16    LocalDateTime lastMatchAt = stats != null ? stats.lastMatchAt() : null;
17    String name = profile != null ? profile.getName() : null;
18    String surname = profile != null ? profile.getSurname() : null;
19    String nickname = profile != null ? profile.getNickname() : null;
20    String language = profile != null ? profile.getLanguage() : null;
21    return new PlayerBioData(
22        userId,
23        name,
24        surname,
25        nickname,
26        language,
27        matchesPlayed,
28        matchesWon,
29        lastMatchAt
30    );
31 }
```

Listing 4.2: T9 - Costruzione PlayerBioData con fallback su stats (PlayerBioDataService)

4.2.3 T9 – GroqLlmClient.java

Porzione analizzata La porzione selezionata mostra:

- costruzione del prompt con vincolo di lunghezza (max 250 caratteri),
- invio della richiesta HTTP al provider LLM,
- fallback a una bio di default in caso di errori o risposte non conformi. Questo garantisce resilienza e prevedibilità della UX.

```

1 private String buildPrompt(PlayerBioData data) {
2     String nickname = data.nickname() != null ? data.nickname() : "";
3     Integer matchesPlayed = data.matchesPlayed() != null ? data.matchesPlayed()
4         : 0;
5     Integer matchesWon = data.matchesWon() != null ? data.matchesWon() : 0;
6     StringBuilder sb = new StringBuilder();
7     sb.append("Genera una biografia breve (max 250 caratteri) in italiano, tono
8         amichevole da gamer. ");
9     if (!nickname.isBlank()) sb.append("nickname: ").append(nickname).append(".
10         ");
11     sb.append("Partite giocate: ").append(matchesPlayed).append(", partite
12         vinte: ").append(matchesWon).append(". ");
13     sb.append("Mantieni un tono amichevole e da gamer, breve e incisivo.");
14     return sb.toString();
15 }
16
17 @Override
18 public String generateBio(PlayerBioData data) {
19     try {
20         String prompt = buildPrompt(data);
21         Map<String, Object> message = new HashMap<>();
22         message.put("role", "user");
23         message.put("content", prompt);
24         Map<String, Object> body = new HashMap<>();
25         body.put("model", model);
26         body.put("messages", List.of(message));
27         body.put("temperature", 0.7);
28         String requestBody = objectMapper.writeValueAsString(body);
29         HttpRequest request = HttpRequest.newBuilder()
30             .uri(URI.create(apiUrl))
31             .header("Content-Type", "application/json")
32             .header("Authorization", "Bearer " + apiKey)
33             .POST(HttpRequest.BodyPublishers.ofString(requestBody))
34             .build();

```

```
30     HttpResponse<String> response = httpClient.send(request, HttpResponse.  
        BodyHandlers.ofString());  
31  
32     if (response.statusCode() >= 200 && response.statusCode() < 300) {  
33         JsonNode root = objectMapper.readTree(response.body());  
34         JsonNode choices = root.path("choices");  
35         if (choices.isArray() && choices.size() > 0) {  
36             JsonNode contentNode = choices.get(0).path("message").path("  
                content");  
37             if (!contentNode.isMissingNode() && !contentNode.asText().  
                isBlank()) {  
38                 String bio = contentNode.asText().trim();  
39                 if (bio.length() > 250) {  
40                     bio = bio.substring(0, 247) + "...";  
41                 }  
42                 return bio;  
43             }  
44         }  
45         logger.warn("Groq response did not contain expected content field");  
46     } else {  
47         logger.warn("Groq API returned non-success status {}: {}", response.  
            statusCode(), response.body());  
48     }  
49     } catch (Exception e) {  
50         logger.warn("Failed to generate bio from Groq: {}", e.getMessage());  
51     }  
52     return "Giocatore attivo su T4, amante delle sfide. Profilo in  
        aggiornamento.";  
53 }
```

Listing 4.3: T9 - Prompt engineering e fallback bio (GroqLlmClient)

4.2.4 T9 – UserProfileService.java (Nickname uniqueness)

Porzione analizzata L'estratto implementa la regola di unicità del nickname: prima di salvare, il servizio verifica se esiste già un profilo con lo stesso nickname e, in tal caso, solleva una `DuplicateNicknameException`. Questa eccezione verrà poi mappata a HTTP 409 dal `RestExceptionHandler`, permettendo al frontend (T5) di gestire l'errore.

```
1 public UserProfile updateNickname(Long userId, String newNickname) {
2     UserProfile me = getOrCreateByUserId(userId);
3
4     repository.findByNickname(newNickname).ifPresent(other -> {
5         Long otherUserId = other.getUserId();
6         if (otherUserId != null && !otherUserId.equals(userId)) {
7             throw new DuplicateNicknameException("Nickname already taken: " +
8                 newNickname);
9         }
10    });
11
12    me.setNickname(newNickname);
13    return repository.save(me);
14 }
```

Listing 4.4: T9 - Unicità nickname e raising exception (UserProfileService)

4.2.5 T9 – RestExceptionHandler.java (HTTP 409)

Porzione analizzata Questa porzione mostra il mapping dell'eccezione a una risposta HTTP semanticamente corretta: 409 `CONFLICT`. In questo modo il frontend può distinguere un conflitto (nickname già usato) da altri errori generici.

```
1 @ExceptionHandler(DuplicateNicknameException.class)
2 public ResponseEntity<?> handleDuplicateNickname(DuplicateNicknameException ex)
3 {
4     return ResponseEntity.status(HttpStatus.CONFLICT).body(Map.of(
5         "error", "NICKNAME_ALREADY_TAKEN",
6         "message", ex.getMessage()
7     ));
8 }
```

Listing 4.5: T9 - Mapping `DuplicateNicknameException` -> HTTP 409 (`RestExceptionHandler`)

4.2.6 T9 – UserProfileController.java

Porzione analizzata L'estratto seguente mostra la progettazione di endpoint REST dedicati al profilo dell'utente autenticato (*/me*). La scelta riduce la necessità di passare esplicitamente l'identificativo utente dal frontend e consente di centralizzare la logica di validazione e accesso nel servizio.

```
1 @RestController
2 @RequestMapping("/api/profiles")
3 public class UserProfileController {
4
5     @GetMapping("/me")
6     public UserProfileResponseDTO getMyProfile(
7         @RequestHeader("X-Authenticated-UserId") Long userId) {
8         return userProfileService.getProfileByUserId(userId);
9     }
10
11     @PutMapping("/me/nickname")
12     public UserProfileResponseDTO updateNickname(
13         @RequestHeader("X-Authenticated-UserId") Long userId,
14         @RequestBody UpdateNicknameRequest request) {
15         return userProfileService.updateNickname(userId, request.getNickname());
16     }
17 }
```

Listing 4.6: T9 - Endpoint REST per lettura e aggiornamento nickname del profilo autenticato

4.2.7 T23 – PlayerController.java

Porzione analizzata L'estratto evidenzia il ruolo del `PlayerController` come punto di esposizione delle operazioni principali legate al *dominio utente/giocatore* (lettura e aggiornamento di informazioni correlate al player). In particolare, il controller delega la logica al service layer, mantenendo un design coerente con la separazione Controller/Service.

```
1 @RestController
2 @RequestMapping("/player")
3 public class PlayerController {
4
5     private final PlayerService playerService;
6
7     public PlayerController(PlayerService playerService) {
8         this.playerService = playerService;
```

```
9     }
10
11     @GetMapping("/{playerId}")
12     public PlayerDTO getPlayer(@PathVariable Long playerId) {
13         return playerService.getPlayerById(playerId);
14     }
15 }
```

Listing 4.7: T23 - Esempio di endpoint esposto da PlayerController e delega a PlayerService

4.2.8 T23 – UserProfileRepository.java

Porzione analizzata Questo estratto mostra l'uso del pattern Repository (Spring Data JPA) per accedere ai dati persistiti da T23. È rilevante perché chiarisce come T23 continui a mantenere la propria persistenza (autenticazione/social/compatibilità legacy), senza accedere al DB di T9.

```
1 @Repository
2 public interface UserProfileRepository extends JpaRepository<UserProfile,
3     Integer> {
4
5     Optional<UserProfile> findByPlayerId(Long playerId);
6
7     Optional<UserProfile> findByEmail(String email);
8 }
```

Listing 4.8: T23 - Repository per accesso ai profili nel DB di T23

4.2.9 T23 – PlayerService.java

Porzione analizzata L'estratto evidenzia un tipico flusso di service layer: recupero dal repository, validazione e mapping verso DTO. Questa struttura riduce l'accoppiamento tra controller e persistenza e facilita refactoring/test.

```
1 @Service
2 public class PlayerService {
3
4     private final UserProfileRepository userProfileRepository;
5
6     public PlayerService(UserProfileRepository userProfileRepository) {
7         this.userProfileRepository = userProfileRepository;
8     }
9 }
```

```
10 public PlayerDTO getPlayerById(Long playerId) {
11     UserProfile profile = userProfileRepository.findByPlayerId(playerId)
12         .orElseThrow(() -> new RuntimeException("Player not found"));
13     return new PlayerDTO(profile.getPlayerId(), profile.getName(), profile.
14         getSurname(), profile.getEmail());
15 }
```

Listing 4.9: T23 - Esempio di logica applicativa in PlayerService (recupero e mapping)

4.2.10 T23 – AuthService.java

Porzione analizzata L'estratto evidenzia l'integrazione tra T23 e T9 durante registrazione/login: T23 tenta l'inizializzazione/sincronizzazione del profilo su T9 tramite chiamata HTTP. La gestione in *try/catch* implementa una strategia *best-effort*, evitando che un malfunzionamento di T9 blocchi l'autenticazione dell'utente.

```
1 try {
2     restTemplate.postForEntity(
3         t9ProfileBaseUrl + "/" + playerId + "/init",
4         initProfileRequest,
5         Void.class
6     );
7 } catch (Exception e) {
8     logger.warn("Profile initialization on T9 failed", e);
9 }
```

Listing 4.10: T23 - Inizializzazione del profilo su T9 in modalita' best-effort

4.2.11 T5 – UserProfileController.java

Porzione analizzata Il seguente estratto mostra il routing lato UI verso le pagine di profilo: la vista del profilo personale (*/profile*) e del profilo di un altro utente (*/friend/{id}*). Il controller prepara solamente i parametri minimi per la view, mentre i dati vengono caricati lato client tramite chiamate REST.

```
1 @GetMapping("/profile")
2 public String profilePage() {
3     return "t9_profile";
4 }
5
6 @GetMapping("/friend/{playerId}")
7 public String friendProfile(@PathVariable Long playerId, Model model) {
8     model.addAttribute("friendId", playerId);
9 }
```

```
9     return "friend_profile";
10 }
```

Listing 4.11: T5 - Routing verso pagine profilo e profilo amico

4.2.12 T5 – friend_profile.js

Porzione analizzata L'estratto mostra come la UI carichi il profilo di un altro utente tramite T9 e, in parallelo, recuperi lo stato social (followers/following/isFollowing) tramite endpoint esposti da T5 (/follow/{id}), che fungono da proxy verso T23. Questo evidenzia l'aggregazione a livello UI, senza introdurre dipendenze dirette T9 → T23.

```
1 const API_BASE = "/api/api/profiles";
2
3 const friendId = window.__FRIEND_ID__;
4
5 // Profilo (T9 via Gateway)
6 fetch(`${API_BASE}/${friendId}`, { credentials: "include" })
7   .then(r => r.json())
8   .then(p => renderFriendProfile(p));
9
10 // Social (T5 -> T23 via ServiceManager)
11 fetch(`/follow/${friendId}`, { credentials: "include" })
12   .then(r => r.json())
13   .then(s => renderSocialBox(s));
```

Listing 4.12: T5 - Caricamento profilo amico (T9) e info social (T5 proxy)

4.2.13 T5 – profile.js

Porzione analizzata L'estratto illustra il recupero del profilo dell'utente autenticato tramite chiamata `fetch` verso il gateway (/api/...), includendo i cookie di sessione (`credentials: "include"`). In questo modo il frontend resta disaccoppiato dalla topologia dei microservizi e demanda il routing al gateway.

```
1 const API_BASE = "/api/api/profiles";
2
3 fetch(`${API_BASE}/me`, { credentials: "include" })
4   .then(res => res.json())
5   .then(profile => renderProfile(profile));
```

Listing 4.13: T5 - Caricamento profilo tramite chiamata REST verso T9 (via Gateway)

4.2.14 T5 – fragments/Navbar.html (link alla pagina profilo)

Porzione analizzata L'estratto evidenzia la modifica della navbar per rendere la navigazione verso la pagina profilo esplicita e coerente, sostituendo accessi frammentati (es. pulsanti isolati per achievements) con un entry-point unico (/profile).

```
1 <nav>
2   <!-- ... altri elementi ... -->
3   <ul class="dropdown-menu">
4     <li><a class="dropdown-item" href="/profile">Profilo</a></li>
5     <!-- elementi legacy deprecati relativi agli achievements -->
6   </ul>
7 </nav>
```

Listing 4.14: T5 - Navbar: collegamento alla pagina profilo unificata

4.2.15 T5 – friend_profile.html

Porzione analizzata Questa porzione mostra come la view passi al JavaScript l'identificativo dell'amico tramite variabile globale, consentendo allo script di effettuare le chiamate REST necessarie senza accoppiare la pagina alla logica backend.

```
1 <script>
2   window.__FRIEND_ID__ = /*[[${friendId}]]*/ 0;
3 </script>
4 <script src="/T5/js/friend_profile.js"></script>
```

Listing 4.15: T5 - friend_profile.html: passaggio friendId allo script

4.2.16 T5 – t9_profile.html

Porzione analizzata Il template della pagina profilo personale include gli script necessari per caricare e modificare dinamicamente i dati tramite le API di T9. L'approccio riduce la logica server-side nella UI e centralizza i dati nel servizio profilo.

```
1 <div id="profile-container"></div>
2
3 <script src="/T5/js/profile.js"></script>
```

Listing 4.16: T5 - t9_profile.html: inclusione script di caricamento profilo

Capitolo 5

Test

Questo capitolo descrive i test eseguiti per verificare la correttezza delle funzionalità implementate e l'integrazione tra i microservizi coinvolti. I test sono stati condotti con particolare attenzione ai flussi end-to-end (Registrazione/Login → Creazione profilo → Visualizzazione UI), alla robustezza delle comunicazioni inter-servizio e alla gestione degli errori.

5.1 Ambito dei Test

I test hanno coperto i seguenti livelli:

- **Test API:** verifica delle principali rotte REST per lettura e modifica del profilo, gestione errori e validazioni.
- **Unit Test:** verifica del corretto funzionamento di una singola funzione.
- **Test di integrazione:** verifica interazione tra diversi moduli o componenti del sistema.
- **Test End-to-End:** simulazione di scenari utente.
- **Test di routing (API Gateway / UI Gateway):** verifica del corretto instradamento delle richieste da browser verso i servizi.

5.2 Strumenti Utilizzati

- **Postman:** test manuali delle API REST (GET/PUT/POST) e validazione delle risposte.
- **Browser + DevTools:** verifica flusso UI e controllo delle chiamate di rete (Network tab), cookie e status code.
- **Log dei servizi (Spring Boot/Docker):** verifica delle chiamate inter-servizio e diagnosi di errori.

5.3 Casi di Test Principali

5.3.1 Test API – Microservizio T9

Tabella 5.1: Casi di test principali sulle API di T9

ID	Scenario	Output Atteso	Esito
T9-01	GET profilo autenticato (/api/profiles/me)	HTTP 200 + dati profilo coerenti	OK
T9-02	PUT nickname valido (/me/nickname)	HTTP 200 + nickname aggiornato in DB	OK
T9-03	PUT nickname duplicato	HTTP 409 (Conflict) + messaggio errore	OK
T9-04	POST generate bio (/me/generate-bio)	HTTP 200 + bio valorizzata (o fallback)	OK
T9-05	Richiesta senza autenticazione header	HTTP 401 (Unauthorized)	OK

5.3.2 Unit Test

Tabella 5.2: Principali Unit Test di T9

ID	Scenario	Output Atteso	Esito
UT-01	Inizializzazione nuovo profilo	Profilo creato con nickname di default e biografia vuota	OK
UT-02	Aggiornamento nickname valido	Nickname aggiornato correttamente nel profilo	OK
UT-03	Aggiornamento nickname non valido	Eccezione di validazione sollevata: HTTP 401 (Unauthorized)	OK
UT-04	Aggiornamento nickname duplicato	Eccezione di conflitto (nickname già esistente): HTTP 409 (Conflict)	OK
UT-05	Generazione biografia con servizio AI disponibile	Biografia generata e associata al profilo	OK

5.3.3 Test di Integrazione – T23 → T9

Tabella 5.3: Casi di test di integrazione tra T23 e T9

ID	Scenario	Output Atteso	Esito
INT-01	Registrazione utente con T9 attivo	Profilo creato su T9 (nickname default, bio vuota)	OK
INT-02	Login utente con profilo non presente su T9	Profilo creato/inizializzato al login (<i>best-effort</i>)	OK
INT-03	Registrazione con T9 non disponibile	Registrazione non bloccata su T23 + warning nei log	OK

5.3.4 Test di Integrazione – T9 → LLM

Tabella 5.4: Casi di test di integrazione tra T9 e LLM

ID	Scenario	Output Atteso	Esito
INT-04	Generazione bio con LLM disponibile	Bio generata e salvata nel profilo	OK
INT-05	Timeout/errore LLM	Bio non aggiornata, messaggio di fallback predefinito (<i>best-effort</i>)	OK

5.3.5 Test End-to-End

Tabella 5.5: Casi di test end-to-end

ID	Scenario	Output Atteso	Esito
E2E-01	Accesso pagina profilo (/profile)	Dati caricati correttamente da T9	OK
E2E-02	Modifica nickname/bio da UI	Aggiornamento immediato UI + persistenza su T9	OK
E2E-03	Generazione bio AI	Bio aggiornata in UI (o fallback)	OK
E2E-04	Apertura profilo amico (/friend/{id})	Profilo amico caricato da T9	OK
E2E-05	Follow/unfollow amico	Toggle stato + conteggi coerenti	OK
E2E-06	Ricerca utenti (Search-Friend)	Risultati corretti e paginazione	OK

5.4 Problemi Rilevati e Correzioni

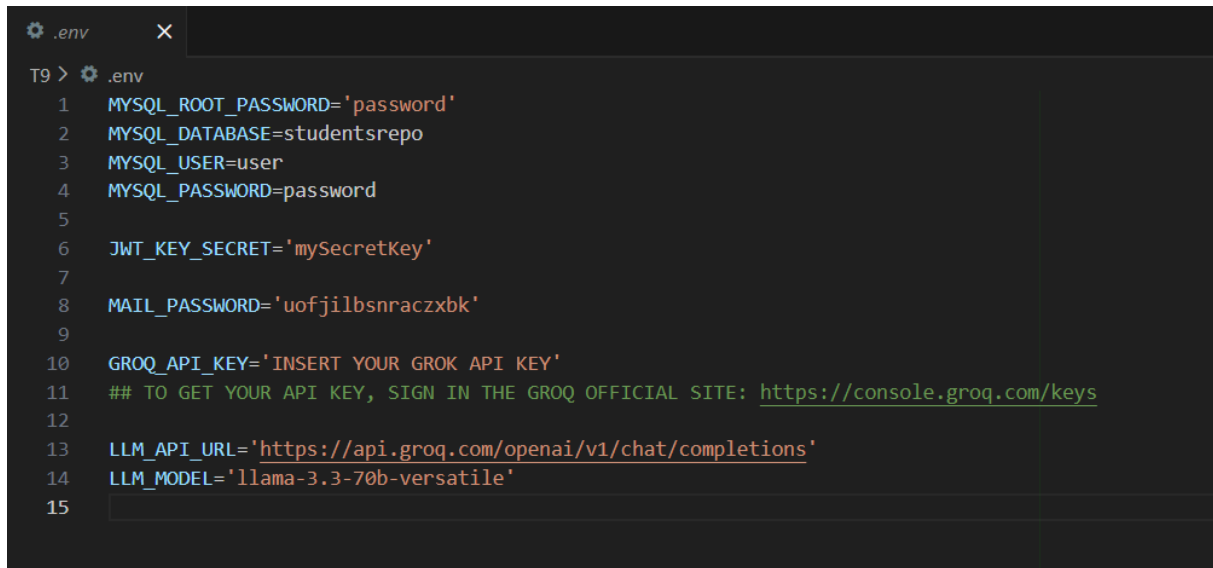
Durante i test sono emerse alcune criticità, risolte nel corso dello sviluppo:

- **Conflitti DB/Autenticazione su copia di T23:** l'approccio iniziale di sviluppo su copia ha evidenziato dipendenze non banali (login e gestione DB). La soluzione è stata introdurre un microservizio dedicato (T9) e integrare tramite API ben definite.
- **Gestione nickname duplicato:** introdotta validazione lato backend (HTTP 409) e gestione esplicita dell'errore lato frontend.
- **Robustezza generazione bio:** previsto fallback in caso di indisponibilità del provider LLM (es. API key mancante o errore rete).

5.5 Limitazioni Note

Nonostante le funzionalità principali siano state implementate e verificate correttamente, sono state individuate alcune limitazioni note, legate a scelte progettuali consapevoli e a vincoli di sicurezza.

- **Dipendenza da servizio LLM esterno per la generazione della bio:** la funzionalità di generazione automatica della biografia del giocatore si basa su un servizio di intelligenza artificiale esterno (Groq). Per motivi di sicurezza, la chiave di accesso al servizio non è inclusa nel repository del progetto. In assenza della chiave configurata nel file `.env`, il sistema utilizza un meccanismo di *fallback* che assegna all'utente una bio di default. Il corretto funzionamento della generazione della bio è quindi subordinato alla configurazione dell'ambiente di esecuzione da parte dell'amministratore, che deve fornire la chiave API nel file `.env`.
- **Funzionalità di profilo pubblico/privato non ancora attiva:** è stato introdotto un meccanismo di toggle che consente all'utente di impostare il proprio profilo come pubblico o privato. Tuttavia, allo stato attuale, tale impostazione non influisce sul comportamento del sistema: la logica applicativa che dovrebbe limitare la visibilità del profilo in base a questo flag non è stata implementata. La funzionalità è quindi presente a livello di interfaccia e modello dati, ma non ancora operativa.

A screenshot of a code editor window with a dark theme. The title bar at the top shows a gear icon, the filename ".env", and a close button (X). The editor content shows a list of environment variables. Line numbers 1 through 15 are visible on the left. The variables are: 1. MYSQL_ROOT_PASSWORD='password', 2. MYSQL_DATABASE=studentsrepo, 3. MYSQL_USER=user, 4. MYSQL_PASSWORD=password, 5. (empty line), 6. JWT_KEY_SECRET='mySecretKey', 7. (empty line), 8. MAIL_PASSWORD='uofjilbsnraczbk', 9. (empty line), 10. GROQ_API_KEY='INSERT YOUR GROK API KEY', 11. ## TO GET YOUR API KEY, SIGN IN THE GROQ OFFICIAL SITE: <https://console.groq.com/keys>, 12. (empty line), 13. LLM_API_URL='https://api.groq.com/openai/v1/chat/completions', 14. LLM_MODEL='llama-3.3-70b-versatile', 15. (empty line).

```
.env
1  MYSQL_ROOT_PASSWORD='password'
2  MYSQL_DATABASE=studentsrepo
3  MYSQL_USER=user
4  MYSQL_PASSWORD=password
5
6  JWT_KEY_SECRET='mySecretKey'
7
8  MAIL_PASSWORD='uofjilbsnraczbk'
9
10 GROQ_API_KEY='INSERT YOUR GROK API KEY'
11 ## TO GET YOUR API KEY, SIGN IN THE GROQ OFFICIAL SITE: https://console.groq.com/keys
12
13 LLM_API_URL='https://api.groq.com/openai/v1/chat/completions'
14 LLM_MODEL='llama-3.3-70b-versatile'
15
```

Figura 5.1: Attuale file `.env` di Progetto

Capitolo 6

Sviluppi Futuri

L'architettura del microservizio T9 è stata progettata per supportare evoluzioni future focalizzate sull'identità digitale del giocatore:

- **Sezione Notifiche:** Implementazione di un **sistema di avvisi in tempo reale** per le interazioni gestite direttamente da T9 (es. nuovi follower). Il profilo diventerebbe l'hub centralizzato per la consultazione dello storico delle notifiche, migliorando l'accessibilità delle informazioni.
- **Group's Identity:** Integrazione dei dati provenienti dai microservizi di gestione gruppi per **visualizzare** sulla pagina profilo **l'appartenenza ad un team**. Questo include l'esposizione di achievements collettivi, rafforzando il legame tra identità individuale e sociale.
- **Badges:** Introduzione di un sistema di "vetrina" che permetta all'utente di selezionare e fissare sul profilo i **badge più prestigiosi** tra quelli ottenuti (gestiti da T5). Questa funzionalità massimizza la personalizzazione e l'engagement, permettendo all'utente di definire la propria immagine pubblica nel sistema.